

A Laboratory Manual for

# **DS using Python Skill based Lab (ITL605) Semester - VI**

Bachelor's Degree in Information Technology



The Bombay Salesian Society's  
**DON BOSCO INSTITUTE OF TECHNOLOGY**  
*(An Autonomous Institute affiliated to University of Mumbai)*  
Department of Information Technology

Academic Year 2025 – 2026

## Index

| Sr. No. | Title of Experiment                                       | CO Mapping | Date of Performance | Date of Submission | Page No. | Grade/Marks | Sign |
|---------|-----------------------------------------------------------|------------|---------------------|--------------------|----------|-------------|------|
| 1       | Experiment 1 - Exploratory Data Analysis (EDA)            |            |                     |                    |          |             |      |
| 2       | Experiment 2 - Data Visualization                         |            |                     |                    |          |             |      |
| 3       | Experiment 3 - Correlation Analysis (Pearson Coefficient) |            |                     |                    |          |             |      |
| 4       | Experiment 4 - Supervised Learning                        |            |                     |                    |          |             |      |
| 5       | Experiment 5 - Unsupervised Learning                      |            |                     |                    |          |             |      |
| 6       | Experiment 6 - Hypothesis Testing                         |            |                     |                    |          |             |      |
| 7       | Experiment 7 - A/B Testing                                |            |                     |                    |          |             |      |
| 8       | Experiment 8 - ANOVA Test                                 |            |                     |                    |          |             |      |
| 9       | Experiment 9 - Data Distribution                          |            |                     |                    |          |             |      |

Subject In charge  
(Prof. Shiv Negi)

The Bombay Salesian Society's **DON BOSCO INSTITUTE OF TECHNOLOGY**  
*(An Autonomous Institute affiliated to University of Mumbai)*  
 Department of Information Technology

| Experiment No. & Title                                                   | Figure No.     | Figure Title                                            | Page no : |
|--------------------------------------------------------------------------|----------------|---------------------------------------------------------|-----------|
| <b>Experiment 1 —<br/>Exploratory Data Analysis</b>                      |                |                                                         |           |
|                                                                          | <b>Fig 1.1</b> | <b>Descriptive Summary</b>                              |           |
|                                                                          | <b>Fig 1.2</b> | <b>Before and After Null and Missing Value Handling</b> |           |
| <b>Experiment 2 — Data Visualization</b>                                 |                |                                                         |           |
|                                                                          | <b>Fig 2.1</b> | <b>Grouped Bar Chart and Stacked Bar Chart</b>          |           |
|                                                                          | <b>Fig 2.2</b> | <b>Distribution Chart</b>                               |           |
|                                                                          | <b>Fig 2.3</b> | <b>Box Plot</b>                                         |           |
|                                                                          | <b>Fig 2.4</b> | <b>Districts by Total Aadhar Enrollment</b>             |           |
| <b>Experiment 3 —<br/>Correlation Analysis<br/>(Pearson Coefficient)</b> |                |                                                         |           |
|                                                                          | <b>Fig 3.1</b> | <b>Data Creation and I/O</b>                            |           |
|                                                                          | <b>Fig 3.2</b> | <b>Data Viewing and Inspection</b>                      |           |
|                                                                          | <b>Fig 3.3</b> | <b>Data Cleaning and Preprocessing</b>                  |           |
|                                                                          | <b>Fig 3.4</b> | <b>Filling Missing Numeric Values with Means</b>        |           |
|                                                                          | <b>Fig 3.5</b> | <b>Data Manipulation and Transformation</b>             |           |
|                                                                          | <b>Fig 3.6</b> | <b>Grouping and Aggregation</b>                         |           |
| <b>Experiment 4 — Supervised Learning</b>                                |                |                                                         |           |
|                                                                          | <b>Fig 4.1</b> | <b>Distribution Analysis — Histograms</b>               |           |
|                                                                          | <b>Fig 4.2</b> | <b>Correlation Heatmap</b>                              |           |
|                                                                          | <b>Fig 4.3</b> | <b>Scatter Plot</b>                                     |           |
|                                                                          |                |                                                         |           |

The Bombay Salesian Society's **DON BOSCO INSTITUTE OF TECHNOLOGY**  
*(An Autonomous Institute affiliated to University of Mumbai)*  
 Department of Information Technology

|                                                 |                |                                                               |  |
|-------------------------------------------------|----------------|---------------------------------------------------------------|--|
| <b>Experiment 5 —<br/>Unsupervised Learning</b> |                |                                                               |  |
|                                                 | <b>Fig 5.1</b> | <b>K-Means Clusters Visualized with PCA</b>                   |  |
|                                                 | <b>Fig 5.2</b> | <b>Elbow Method for K-Means</b>                               |  |
| <b>Experiment 6 — Hypothesis Testing</b>        |                |                                                               |  |
|                                                 | <b>Fig 6.1</b> | <b>Hypothesis 1 — Sleep &amp; Memory</b>                      |  |
|                                                 | <b>Fig 6.2</b> | <b>Hypothesis 2 — Fertilizer &amp; Crop Yield</b>             |  |
|                                                 | <b>Fig 6.3</b> | <b>Hypothesis 3 — Noise-Cancelling Headphones &amp; Focus</b> |  |
| <b>Experiment 7 — A/B Testing</b>               |                |                                                               |  |
|                                                 | <b>Fig 7.1</b> | <b>YouTube Thumbnail Dataset</b>                              |  |
|                                                 | <b>Fig 7.2</b> | <b>Webpage Design Dataset</b>                                 |  |
|                                                 | <b>Fig 7.3</b> | <b>YouTube Thumbnail A/B Test</b>                             |  |
|                                                 | <b>Fig 7.4</b> | <b>Webpage Design A/B Test</b>                                |  |
| <b>Experiment 8 — ANOVA</b>                     |                |                                                               |  |
|                                                 | <b>Fig 8.1</b> | <b>Mean Scores Bar Chart</b>                                  |  |
|                                                 | <b>Fig 8.2</b> | <b>ANOVA Result Output from Console</b>                       |  |
| <b>Experiment 9 — Data Distribution</b>         |                |                                                               |  |
|                                                 | <b>Fig 9.1</b> | <b>Normal Distribution Graph</b>                              |  |
|                                                 | <b>Fig 9.2</b> | <b>Uniform Distribution Graph</b>                             |  |
|                                                 | <b>Fig 9.3</b> | <b>Binomial Distribution Graph</b>                            |  |
|                                                 | <b>Fig 9.4</b> | <b>Poisson Distribution Graph</b>                             |  |
|                                                 | <b>Fig 9.5</b> | <b>Exponential Distribution Graph</b>                         |  |



## Certificate

This is to certify that Mr **Hanchate Prem Vijay** has successfully completed the Term work in the subject of **DS using Python Skill based Lab** during **TE Semester-VI** in the Academic Year **2025 – 2026**.

Faculty In charge  
**(Prof. Shiv Negi)**

# Experiment 1 — Exploratory Data Analysis (EDA)

**Name : Prem V Hanchate**

**Roll no: 68 Class: TE/IT**

---

## Aim

To perform Exploratory Data Analysis (EDA) on the Aadhar Enrollment dataset to understand its structure, detect and treat missing values, and summarize key statistical properties using descriptive measures such as mean, standard deviation, min, max, and quartiles.

---

## Theory

Exploratory Data Analysis (EDA) is the process of systematically examining a dataset before applying any formal modeling or statistical inference. It helps in understanding the nature of data, finding patterns, spotting anomalies, and checking assumptions.

**1. Data Loading and Preview:** The dataset is loaded into a pandas DataFrame from an external file (Excel/CSV). The head() function is used to display the first few rows, giving an initial overview of the data — its columns, values, and general structure.

**2. Dataset Dimensions:** Understanding the size of the dataset is fundamental. The shape attribute returns the number of rows (records) and columns (features). For the Aadhar dataset, rows represent individual enrollment records and columns represent attributes such as demographic details, registration status, and enrollment statistics.

**3. Descriptive Statistics:** The describe() function generates a statistical summary for all numeric columns. Key measures include:

- **Mean** — Average value of a column, representing central tendency.
- **Standard Deviation** — Measures how spread out the values are from the mean.
- **Min / Max** — The smallest and largest values observed in a column.
- **Quartiles (25%, 50%, 75%)** — Divide the data into four equal parts; the 50th percentile is the median.

These statistics give a quick sense of the scale, range, and distribution of each numeric attribute.

**4. Data Type Information:** The `info()` function provides column-wise details including data types (int, float, object), non-null counts, and memory usage. This helps identify columns that may need type conversion or special handling.

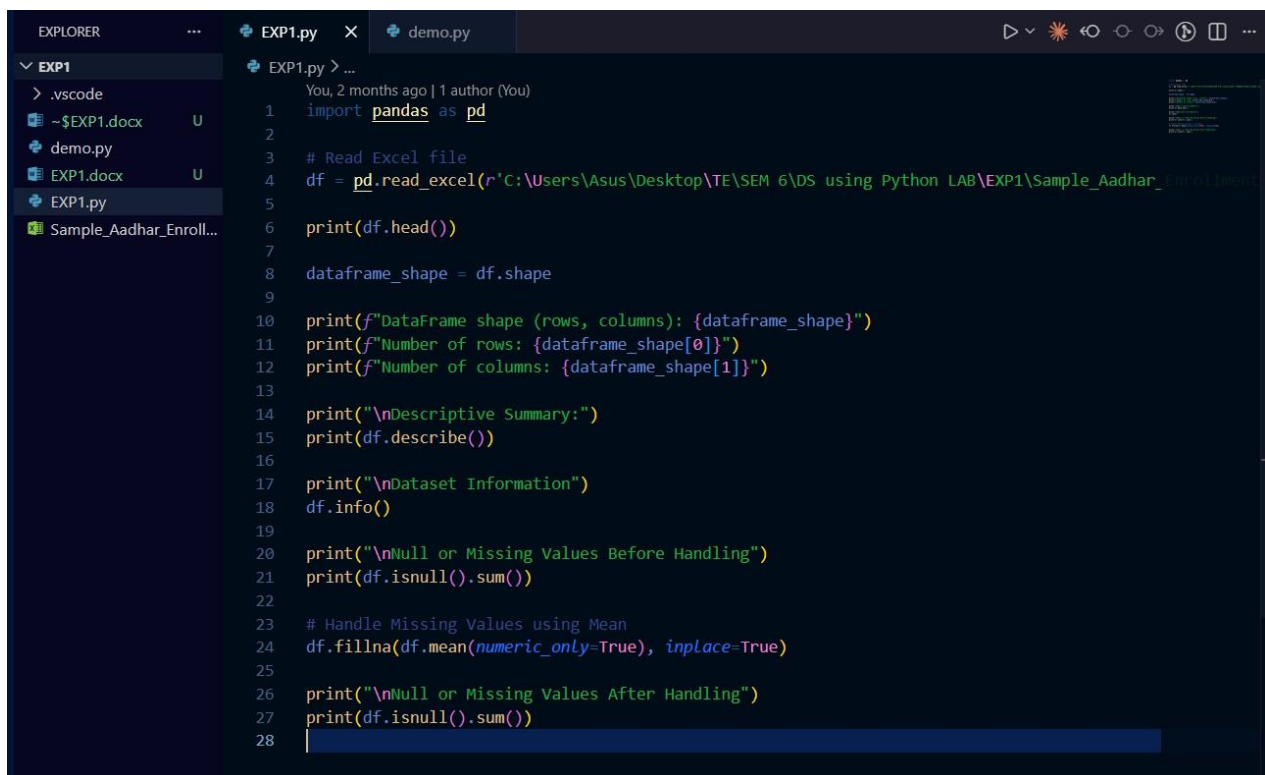
**5. Missing Value Detection:** Missing values (NaN) are common in real-world datasets and can arise from incomplete data entry, system errors, or unavailable records. The `isnull().sum()` function returns the count of missing values per column, helping identify which columns require treatment.

**6. Missing Value Treatment (Imputation):** Once missing values are detected, they are filled using the `fillna()` method. For numeric columns, the mean of the respective column is used for imputation. Mean imputation is a standard technique that preserves the overall distribution without drastically altering summary statistics.

**7. Verification:** After imputation, `isnull().sum()` is run again to confirm that no missing values remain in the dataset, ensuring data quality before any further analysis.

---

## Execution

A screenshot of a Visual Studio Code editor window. The Explorer sidebar on the left shows a project named 'EXP1' with files like '.vscode', '~\$EXP1.docx', 'demo.py', 'EXP1.docx', 'EXP1.py', and 'Sample\_Aadhar\_Enroll...'. The main editor area shows the 'EXP1.py' file with the following Python code:

```
1 import pandas as pd
2
3 # Read Excel file
4 df = pd.read_excel(r'C:\Users\Asus\Desktop\TE\SEM 6\DS using Python LAB\EXP1\Sample_Aadhar_Enrollment.xlsx')
5
6 print(df.head())
7
8 dataframe_shape = df.shape
9
10 print(f"DataFrame shape (rows, columns): {dataframe_shape}")
11 print(f"Number of rows: {dataframe_shape[0]}")
12 print(f"Number of columns: {dataframe_shape[1]}")
13
14 print("\nDescriptive Summary:")
15 print(df.describe())
16
17 print("\nDataset Information")
18 df.info()
19
20 print("\nNull or Missing Values Before Handling")
21 print(df.isnull().sum())
22
23 # Handle Missing Values using Mean
24 df.fillna(df.mean(numeric_only=True), inplace=True)
25
26 print("\nNull or Missing Values After Handling")
27 print(df.isnull().sum())
28
```

---

## Output

```

EXPLORER  ...  PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
EXP1
> .vscode
~$EXP1.docx  U
demo.py
EXP1.docx  U
EXP1.py
Sample_Aadhar_Enroll...

PS C:\Users\Asus\Desktop\TE\SEM 6\DS using Python LAB\EXP1> python EXP1.py

   Date                State      district      pincode  age_0_5  age_5_17  age_18_greater
0  2025-02-03 00:00:00  Meghalaya  East Khasi Hills  793121.0   11.0      61.0      37.0
1  2025-09-03 00:00:00  Karnataka  Bengaluru Urban  560043.0   14.0      33.0      39.0
2  2025-09-03 00:00:00  Uttar Pradesh  Kanpur Nagar  208001.0   29.0      82.0      12.0
3  2025-09-03 00:00:00  Uttar Pradesh  Aligarh      202133.0   62.0      29.0      15.0
4  2025-09-03 00:00:00  Karnataka  Bengaluru Urban  560016.0   14.0      16.0      21.0

DataFrame shape (rows, columns): (99, 7)
Number of rows: 99
Number of columns: 7

Descriptive Summary:
      pincode      age_0_5      age_5_17      age_18_greater
count  96.000000  94.000000  94.000000  95.000000
mean  530493.010417  29.808511  49.117021  21.821053
std   266900.060688  23.710746  44.224466  20.114025
min   110018.000000  10.000000  10.000000  10.000000
25%   265808.000000  14.250000  16.250000  13.000000
50%   560035.500000  20.000000  33.500000  17.000000
75%   783570.000000  35.000000  59.500000  23.500000
max   847421.000000  134.000000  249.000000  148.000000

Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99 entries, 0 to 98
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        99 non-null    object
1   State       98 non-null    object
2   district    94 non-null    object
3   pincode     96 non-null    float64
4   age_0_5     94 non-null    float64
5   age_5_17    94 non-null    float64
6   age_18_greater  95 non-null    float64
dtypes: float64(4), object(3)

```

Fig 1.1 : Descriptive Summary

```

EXPLORER  ...  PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
EXP1
> .vscode
~$EXP1.docx  U
demo.py
EXP1.docx  U
EXP1.py
Sample_Aadhar_Enroll...

PS C:\Users\Asus\Desktop\TE\SEM 6\DS using Python LAB\EXP1> python EXP1.py

Null or Missing Values Before Handling
Date      0
State     1
district  5
pincode   3
age_0_5   5
age_5_17  5
age_18_greater  4
dtype: int64

Null or Missing Values After Handling
Date      0
State     1
district  5
pincode   0
age_0_5   0
age_5_17  0
age_18_greater  0
dtype: int64
PS C:\Users\Asus\Desktop\TE\SEM 6\DS using Python LAB\EXP1>

```

Fig 1.2 : Before and After Null and Missing Value Handling

## Conclusion

In this experiment, Exploratory Data Analysis (EDA) was performed on the Aadhar Enrollment dataset. The dataset structure, data types, and statistics were analyzed using functions like `head()`, `shape`, `info()`, and `describe()`. Missing values were identified and handled by replacing them with the mean of numeric columns using `fillna()`. After cleaning, the dataset became free of null values and ready for further analysis or machine learning tasks.

## Experiment 2 — Data Visualization

Name : Prem Vijay Hanchate

Roll no : 68    Class : TE/IT

---

### Aim

To create meaningful visualizations using bar charts and box plots on the Aadhar Enrollment dataset to represent patterns, trends, and distributions across states, districts, and age groups, and to interpret the insights derived from them.

---

### Theory

Data Visualization is the graphical representation of data and information. It transforms raw numbers into visual formats such as charts, graphs, and plots, making it easier to identify patterns, trends, outliers, and relationships that would otherwise be difficult to detect in tabular form.

**1. Importance of Visualization in Data Science:** After EDA, visualization serves as the next critical step. It enables analysts to communicate findings effectively, validate assumptions, and guide decisions. Python's matplotlib and seaborn libraries are widely used for this purpose.

**2. Bar Chart:** A bar chart represents categorical data with rectangular bars. The height or length of each bar is proportional to the value it represents. Bar charts are ideal for comparing quantities across categories.

- **Grouped Bar Chart** — Displays multiple sub-categories side by side for each main category. Used here to compare age group enrollments (0–5, 5–17, 18+) across different states.
- **Stacked Bar Chart** — Layers sub-categories on top of each other within a single bar, showing both the total and the contribution of each part. Used here to show combined enrollment per state.
- **Simple Bar Chart** — Used to display the Top 10 districts by total enrollment, with value labels on each bar for easy reading.

**3. Box Plot:** A box plot (also called a box-and-whisker plot) summarizes the distribution of a dataset through five statistics:

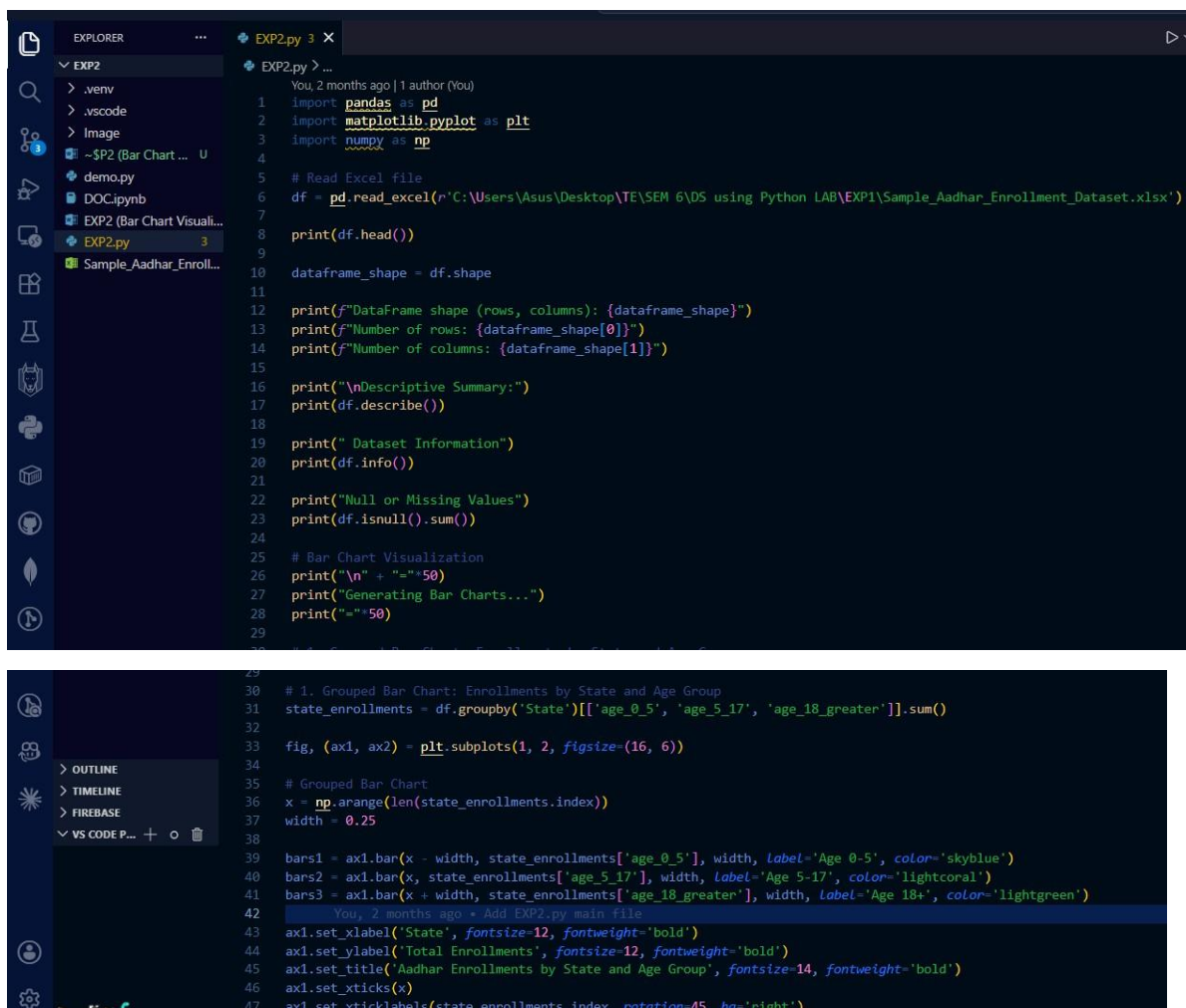
- **Minimum and Maximum** — The ends of the whiskers (excluding outliers).
- **Q1 (25th percentile)** — Lower edge of the box.
- **Median (Q2 / 50th percentile)** — Line inside the box.
- **Q3 (75th percentile)** — Upper edge of the box.
- **IQR (Interquartile Range)** —  $Q3 - Q1$ , representing the middle 50% of data.
- **Outliers** — Points beyond  $1.5 \times \text{IQR}$  from Q1 or Q3, plotted as individual dots.

Box plots are ideal for comparing distributions across multiple groups and identifying skewness and outliers.

**4. Data Aggregation Before Plotting:** Before plotting, raw data is aggregated meaningfully:

- A new column `Total_Enrollment` is computed by summing the three age-group columns (`age_0_5`, `age_5_17`, `age_18_greater`), handling NaN values gracefully.
- Data is grouped by State (summing age-group columns) to compare state-wise enrollment.
- Data is grouped by district (summing `Total_Enrollment`) and sorted to extract the Top 10 districts.

## Execution / Code



```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Read Excel file
6 df = pd.read_excel(r'C:\Users\Asus\Desktop\TE\SEM 6\DS using Python LAB\EXP1\Sample_Aadhar_Enrollment_Dataset.xlsx')
7
8 print(df.head())
9
10 dataframe_shape = df.shape
11
12 print(f'DataFrame shape (rows, columns): {dataframe_shape}')
13 print(f'Number of rows: {dataframe_shape[0]}')
14 print(f'Number of columns: {dataframe_shape[1]}')
15
16 print("\nDescriptive Summary:")
17 print(df.describe())
18
19 print(" Dataset Information")
20 print(df.info())
21
22 print("Null or Missing Values")
23 print(df.isnull().sum())
24
25 # Bar Chart Visualization
26 print("\n" + "-"*50)
27 print("Generating Bar Charts...")
28 print("-"*50)
29
30 # 1. Grouped Bar Chart: Enrollments by State and Age Group
31 state_enrollments = df.groupby('State')[['age_0_5', 'age_5_17', 'age_18_greater']].sum()
32
33 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
34
35 # Grouped Bar Chart
36 x = np.arange(len(state_enrollments.index))
37 width = 0.25
38
39 bars1 = ax1.bar(x - width, state_enrollments['age_0_5'], width, label='Age 0-5', color='skyblue')
40 bars2 = ax1.bar(x, state_enrollments['age_5_17'], width, label='Age 5-17', color='lightcoral')
41 bars3 = ax1.bar(x + width, state_enrollments['age_18_greater'], width, label='Age 18+', color='lightgreen')
42
43 ax1.set_xlabel('State', fontsize=12, fontweight='bold')
44 ax1.set_ylabel('Total Enrollments', fontsize=12, fontweight='bold')
45 ax1.set_title('Aadhar Enrollments by State and Age Group', fontsize=14, fontweight='bold')
46 ax1.set_xticks(x)
47 ax1.set_xticklabels(state_enrollments.index, rotation=45, ha='right')
```

```
EXPLORER
EXP2
> .env
> .vscode
> Image
~$P2 (Bar Chart ... U
demo.py
DOC.ipynb
EXP2 (Bar Chart Visuali...
EXP2.py 3
Sample_Aadhar_Enroll...

OUTLINE
TIMELINE
FIREBASE
VS CODE P... + o

43 ax1.set_xlabel('State', fontsize=12, fontweight='bold')
44 ax1.set_ylabel('Total Enrollments', fontsize=12, fontweight='bold')
45 ax1.set_title('Aadhar Enrollments by State and Age Group', fontsize=14, fontweight='bold')
46 ax1.set_xticks(x)
47 ax1.set_xticklabels(state_enrollments.index, rotation=45, ha='right')
48 ax1.legend()
49 ax1.grid(axis='y', alpha=0.3)
50
51 # Stacked Bar Chart
52 ax2.bar(state_enrollments.index, state_enrollments['age_0_5'], label='Age 0-5', color='skyblue')
53 ax2.bar(state_enrollments.index, state_enrollments['age_5_17'], bottom=state_enrollments['age_0_5'],
54         label='Age 5-17', color='lightcoral')
55 ax2.bar(state_enrollments.index, state_enrollments['age_18_greater'],
56         bottom=state_enrollments['age_0_5'] + state_enrollments['age_5_17'],
57         label='Age 18+', color='lightgreen')
58
59 ax2.set_xlabel('State', fontsize=12, fontweight='bold')
60 ax2.set_ylabel('Total Enrollments', fontsize=12, fontweight='bold')
61 ax2.set_title('Stacked Aadhar Enrollments by State', fontsize=14, fontweight='bold')
62 ax2.set_xticklabels(state_enrollments.index, rotation=45, ha='right')
63 ax2.legend()
64 ax2.grid(axis='y', alpha=0.3)
65
66 plt.tight_layout()
67 plt.savefig('bar_charts_by_state.png', dpi=300, bbox_inches='tight')
68 print("✓ Saved: bar_charts_by_state.png")
69 plt.show()
70
71 # 2. Simple Bar Chart: Top 10 Districts by Total Enrollment
72 df['Total_Enrollment'] = df['age_0_5'].fillna(0) + df['age_5_17'].fillna(0) + df['age_18_greater'].fillna(0)
73 top_districts = df.groupby('district')['Total_Enrollment'].sum().sort_values(ascending=False).head(10)
74
75 plt.figure(figsize=(12, 6))
76 bars = plt.bar(range(len(top_districts)), top_districts.values, color='steelblue', edgecolor='black')
77 plt.xlabel('District', fontsize=12, fontweight='bold')
78 plt.ylabel('Total Enrollments', fontsize=12, fontweight='bold')
79 plt.title('Top 10 Districts by Total Aadhar Enrollments', fontsize=14, fontweight='bold')
80 plt.xticks(range(len(top_districts)), top_districts.index, rotation=45, ha='right')
81 plt.grid(axis='y', alpha=0.3)
82
83 # Add value labels on bars
84 for i, bar in enumerate(bars):
85     height = bar.get_height()
86     plt.text(bar.get_x() + bar.get_width()/2., height,
87             f'{int(height)}',
88             ha='center', va='bottom', fontsize=10, fontweight='bold')
89
```



```
EXPLORER
EXP2
> .venv
> .vscode
> Image
~$P2 (Bar Chart ... U
demo.py
DOC.ipynb
EXP2 (Bar Chart Visuali...
EXP2.py 3
Sample_Aadhar_Enroll...

> OUTLINE
> TIMELINE
> FIREBASE
VS CODE P... + o

EXP2.py 3
89
90 plt.tight_layout()
91 plt.savefig('top_10_districts_bar_chart.png', dpi=300, bbox_inches='tight')
92 print("✓ Saved: top_10_districts_bar_chart.png")
93 plt.show()
94
95 print("\nBar charts displayed successfully!")
96
97 # Box Plot Visualization
98 print("\n" + "="*50)
99 print("Generating Box Plots...")
100 print("="*50)
101
102 # 1. Box Plot: Distribution of Enrollments by Age Groups
103 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
104
105 # Box plot for age group distributions
106 age_data = [df['age_0_5'].dropna(), df['age_5_17'].dropna(), df['age_18_greater'].dropna()]
107 bp1 = ax1.boxplot(age_data, labels=['Age 0-5', 'Age 5-17', 'Age 18+'],
108 | | | | patch_artist=True, notch=True, showmeans=True)
109
110 # Customize box colors
111 colors = ['skyblue', 'lightcoral', 'lightgreen']
112 for patch, color in zip(bp1['boxes'], colors):
113 | patch.set_facecolor(color)
114
115 ax1.set_xlabel('Age Groups', fontsize=12, fontweight='bold')
116 ax1.set_ylabel('Enrollment Count', fontsize=12, fontweight='bold')
117 ax1.set_title('Distribution of Aadhar Enrollments by Age Group', fontsize=14, fontweight='bold')
118 ax1.grid(axis='y', alpha=0.3)
119
120 # Box plot for enrollment distribution by state
121 states = df['State'].dropna().unique()
122 state_data = []
123 state_labels = []
124
125 for state in states:
126 | state_df = df[df['State'] == state]
127 | total_enrollments = state_df['Total_Enrollment'].dropna()
128 | if len(total_enrollments) > 0:
129 | | state_data.append(total_enrollments)
130 | | state_labels.append(state)
131
132 bp2 = ax2.boxplot(state_data, labels=state_labels, patch_artist=True, showmeans=True)
133
134 # Customize box colors with gradient
135 colors2 = plt.cm.viridis(np.linspace(0, 1, len(bp2['boxes'])))
136 for patch, color in zip(bp2['boxes'], colors2):
```



```
EXPLORER
EXP2
EXP2.py 3
EXP2.py > ...
133
134 # Customize box colors with gradient
135 colors2 = plt.cm.viridis(np.linspace(0, 1, len(bp2['boxes'])))
136 for patch, color in zip(bp2['boxes'], colors2):
137     patch.set_facecolor(color)
138
139 ax2.set_xlabel('State', fontsize=12, fontweight='bold')
140 ax2.set_ylabel('Total Enrollment', fontsize=12, fontweight='bold')
141 ax2.set_title('Distribution of Total Enrollments by State', fontsize=14, fontweight='bold')
142 ax2.set_xticklabels(state_labels, rotation=45, ha='right')
143 ax2.grid(axis='y', alpha=0.3)
144
145 plt.tight_layout()
146 plt.savefig('box_plots_distribution.png', dpi=300, bbox_inches='tight')
147 print("✓ Saved: box_plots_distribution.png")
148 plt.show()
149
150 # 2. Single Box Plot: Overall Enrollment Distribution
151 plt.figure(figsize=(10, 6))
152
153 # Prepare data for all age groups combined
154 all_ages_data = [
155     df['age_0_5'].dropna().values,
156     df['age_5_17'].dropna().values,
157     df['age_18_greater'].dropna().values,
158     df['Total_Enrollment'].dropna().values
159 ]
160
161 bp3 = plt.boxplot(all_ages_data, labels=['Age 0-5', 'Age 5-17', 'Age 18+', 'Total'],
162                  patch_artist=True, notch=True, showmeans=True,
163                  meanprops=dict(marker='D', markerfacecolor='red', markersize=8))
164
165 # Customize colors
166 box_colors = ['skyblue', 'lightcoral', 'lightgreen', 'gold']
167 for patch, color in zip(bp3['boxes'], box_colors):
168     patch.set_facecolor(color)
169     patch.set_alpha(0.7)
170
171 plt.xlabel('Enrollment Categories', fontsize=12, fontweight='bold')
172 plt.ylabel('Count', fontsize=12, fontweight='bold')
173 plt.title('Box Plot: Aadhar Enrollment Distribution Analysis\n(Shows Median, Quartiles, and Outliers)',
174          fontsize=14, fontweight='bold')
175 plt.grid(axis='y', alpha=0.3)
176
177 plt.tight_layout()
178 plt.savefig('box_plot_overall_analysis.png', dpi=300, bbox_inches='tight')
179 print("✓ Saved: box_plot_overall_analysis.png")
180 plt.show()
181
182 print("\nBox plots displayed successfully!")
183 print("\nBox Plot Insights:")
184 print("- The box shows the interquartile range (IQR) - middle 50% of data")
185 print("- The line inside the box is the median")
186 print("- The diamond marker shows the mean")
187 print("- Whiskers extend to 1.5 * IQR")
188 print("- Points beyond whiskers are potential outliers")
189
190 print("\n" + "="*50)
191 print("ALL PLOTS SAVED SUCCESSFULLY!")
192 print("="*50)
193 print("\nSaved Files:")
194 print("1. bar_charts_by_state.png")
195 print("2. top_10_districts_bar_chart.png")
196 print("3. box_plots_distribution.png")
197 print("4. box_plot_overall_analysis.png")
198 print("\nYou can now view all plots as image files in the current directory.")
199 print("Simply close each plot window as it appears to continue.")
200
```

```
EXPLORER
EXP2
EXP2.py 3
EXP2.py > ...
181
182 print("\nBox plots displayed successfully!")
183 print("\nBox Plot Insights:")
184 print("- The box shows the interquartile range (IQR) - middle 50% of data")
185 print("- The line inside the box is the median")
186 print("- The diamond marker shows the mean")
187 print("- Whiskers extend to 1.5 * IQR")
188 print("- Points beyond whiskers are potential outliers")
189
190 print("\n" + "="*50)
191 print("ALL PLOTS SAVED SUCCESSFULLY!")
192 print("="*50)
193 print("\nSaved Files:")
194 print("1. bar_charts_by_state.png")
195 print("2. top_10_districts_bar_chart.png")
196 print("3. box_plots_distribution.png")
197 print("4. box_plot_overall_analysis.png")
198 print("\nYou can now view all plots as image files in the current directory.")
199 print("Simply close each plot window as it appears to continue.")
200
```

## Output

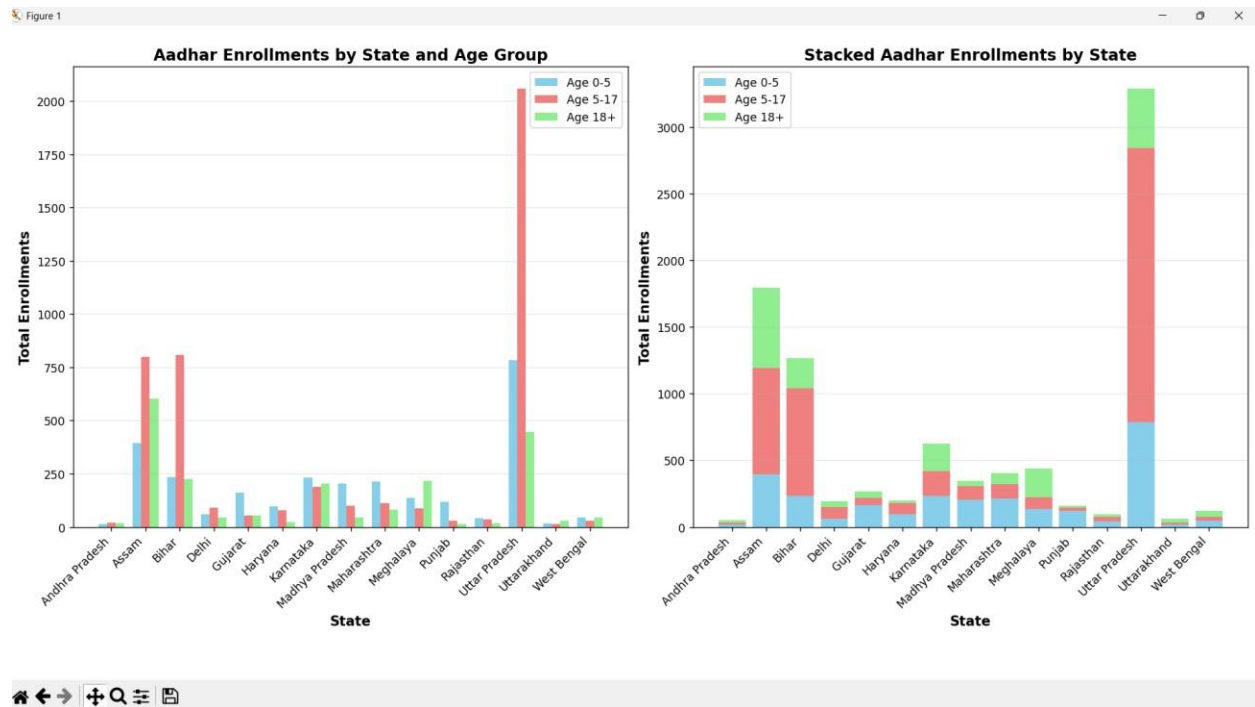


Fig 2.1 : Grouped Bar Chart and Stacked Bar Chart

### Chart's Visualizations Created:

- Grouped Bar Chart (Left panel)**
  - Compares enrollments across different states
  - Shows three age groups side-by-side for each state
  - Uses different colors: Age 0-5 (blue), Age 5-17 (coral), Age 18+ (green)
- Stacked Bar Chart (Right panel)**
  - Shows total enrollments per state
  - Age groups stacked on top of each other
  - Gives a clear view of total enrollment volume per state

The bar charts are ideal for this dataset because they clearly compare categorical data (states/districts) with enrollment counts across different age groups, making it easy to identify patterns and regional differences in Aadhar enrollments.

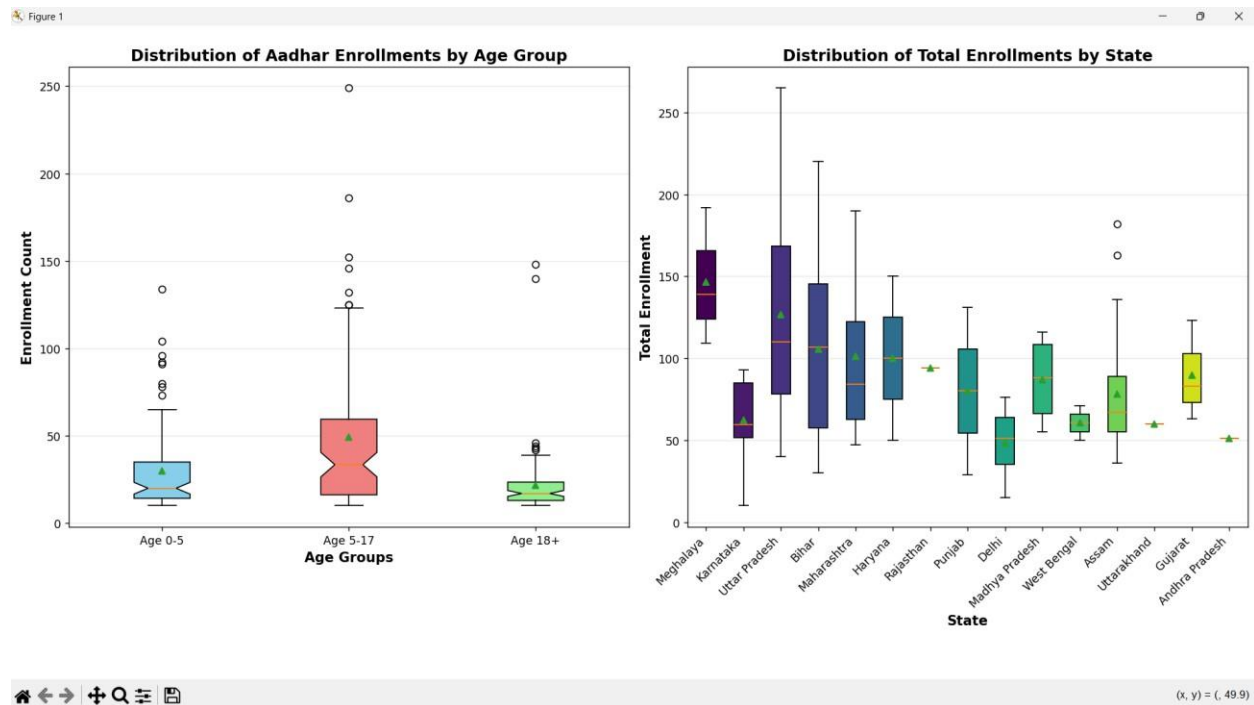


Fig 2.2 : Distribution Chart

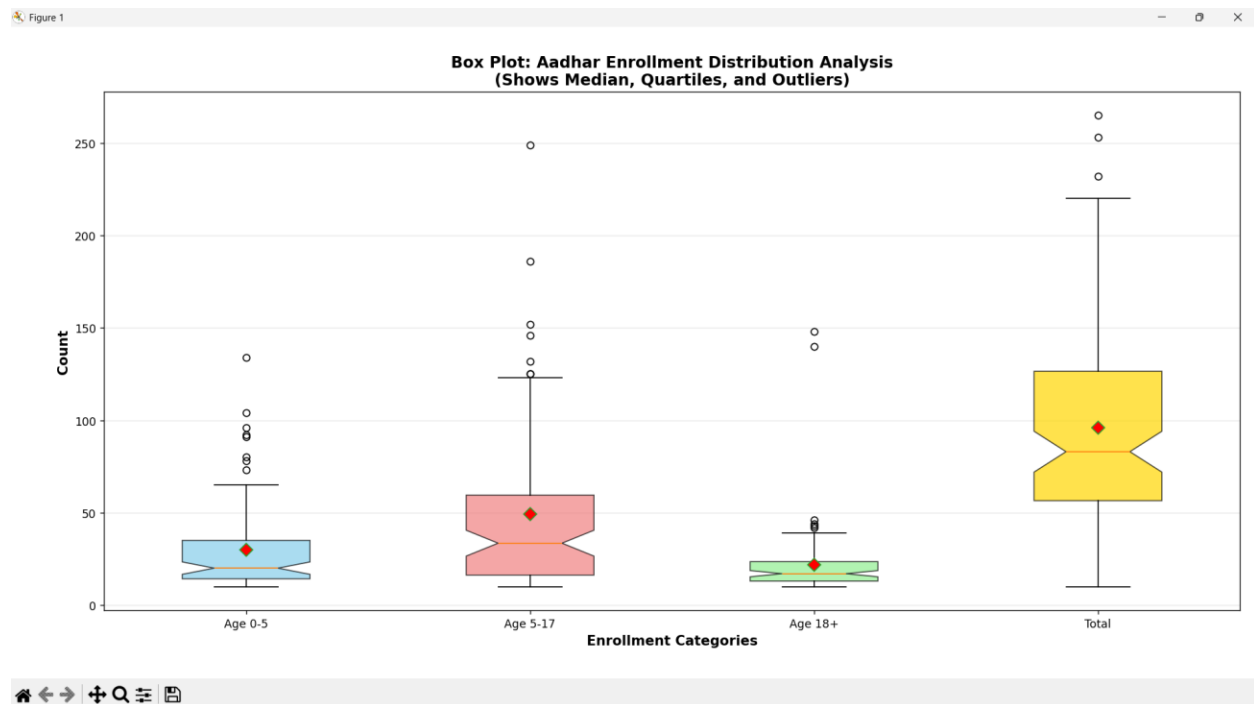


Fig 2.3 : Box Plot

## **High Performing States**

### **1) Uttar Pradesh**

Highest overall enrollment in both graphs

- **Age 5–17** → Extremely high (largest among all states)
- **Age 0–5** → Very high
- **Age 18+** → Moderate but still large compared to many states

Uttar Pradesh is the **top performing state** with maximum Aadhaar enrollment, especially among school-age children.

### **2) Assam**

Second highest performer

- High enrollment in **Age 5–17**
- High enrollment in **Age 18+**
- Good enrollment in **Age 0–5**

Assam shows **balanced and strong performance** across all age groups.

---

## **Equal Performing States**

These states show **balanced but medium-level enrollments**.

### **1) Karnataka**

- Almost equal enrollment in all age groups
- Moderate totals

Shows balanced Aadhaar awareness and participation.

---

## **Low Performing States**

These states show **lowest Aadhaar enrollment totals**.

### **1) Andhra Pradesh**

- Very low enrollments across all age groups

### **2) Uttarakhand**

- Low enrollment in all age groups
  - Still falls in low performance group
- 

## **Age Group Wise Performance Insight**

### Age 0–5 (Children)

- Highest → Uttar Pradesh
- Good → Assam, Bihar
- Lowest → Andhra Pradesh, Jharkhand

### Age 5–17 (School Age)

- Highest → Uttar Pradesh (dominates strongly)
- Second → Assam & Bihar
- Lowest → Andhra Pradesh, Gujarat

### Age 18+

- Highest → Assam
- Moderate → Uttar Pradesh
- Lowest → Andhra Pradesh, Jharkhand

### TOP 10 Districts by Total Aadhar Enrollment

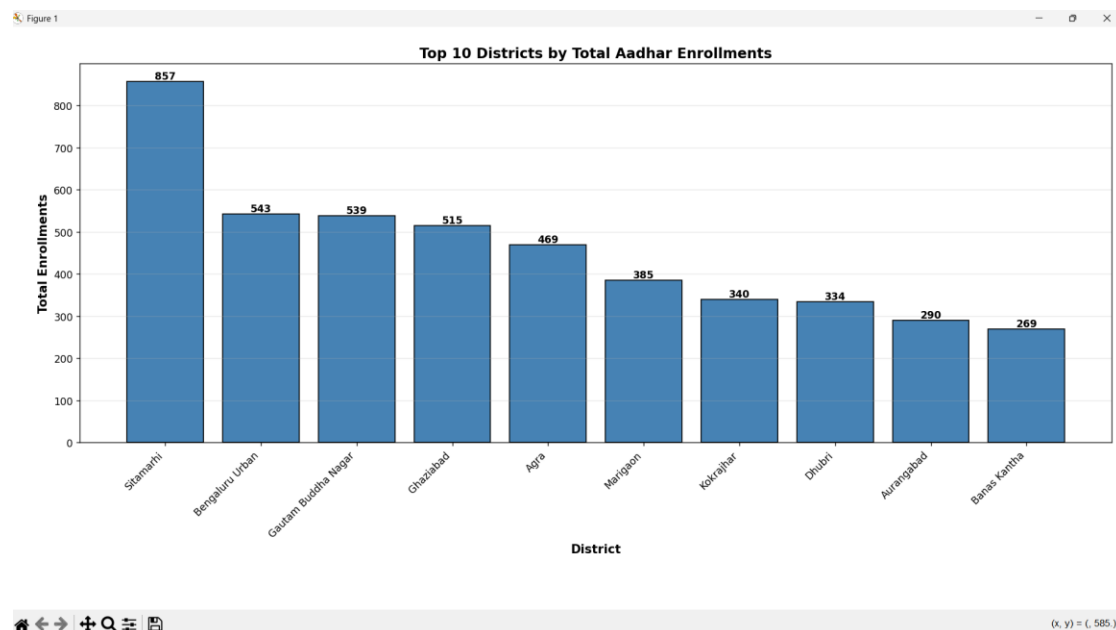


Fig 2.4 : Districts by Total Aadhar Enrollment

## **Conclusion**

In this experiment, data visualization was successfully performed on the Aadhar Enrollment dataset. A new Total\_Enrollment column was derived by summing the three age-group columns. Grouped and stacked bar charts revealed the distribution of enrollments across age groups for each state, highlighting which states had higher participation in specific age brackets. The Top 10 districts bar chart with value labels made it easy to identify the highest-enrolling districts at a glance. Box plots provided insight into the spread, median, IQR, and outliers of enrollment counts across age groups and states. Together, these visualizations offered a comprehensive and interpretable picture of enrollment patterns across India's Aadhar dataset.

# Experiment 3 — Correlation Analysis (Pearson Coefficient)

**Name : Prem Vijay Hanchate**

**Roll no : 68    Class : TE/IT**

## Aim

To perform data wrangling and preprocessing operations on the Aadhar Enrollment dataset using the Pandas library, including handling missing values, removing duplicates, replacing values, applying custom functions, sorting, pivoting, and grouping, in order to prepare the dataset for further analysis.

---

## Theory

**1. Pandas and DataFrames:** Pandas is an open-source Python library built on top of NumPy, designed for data manipulation and analysis. Its primary data structure, the DataFrame, is a two-dimensional, tabular, labeled structure that can hold columns of different data types. It is the most widely used tool for structured data wrangling in Python.

**2. Data Inspection:** Before any manipulation, it is essential to understand the dataset. Key inspection functions include:

- `head()` / `tail()` — View first and last N rows.
- `info()` — Column names, data types, and non-null counts.
- `describe()` — Summary statistics for numeric columns.
- `shape` — Dimensions (rows, columns).
- `columns` — List of all column names.

**3. Missing Value Treatment:** Real-world datasets often contain null or NaN values. The `isnull().sum()` function identifies the count of missing values per column. Treatment strategies include:

- **Numeric columns** — Filling with the column mean using `fillna(mean)` preserves the overall distribution.
- **Categorical/text columns** — Filling with a placeholder such as "Not Provided" avoids data loss while flagging incomplete records.

In this dataset, missing values were found in State (1), district (5), pincode (14), age\_0\_5 (5), age\_5\_17 (5), and age\_18\_greater (4), all of which were handled by filling with the mean or a placeholder.

**4. Duplicate Detection and Removal:** Duplicate rows can skew analysis results. The `duplicated()` function identifies repeated rows, and `drop_duplicates()` removes them. In this experiment, a test set of 101 rows (with 2 intentional duplicates added) was reduced back to 99 unique records after applying `drop_duplicates()`.

**5. Value Replacement:** The `replace()` function is used to substitute existing values with new ones. This is useful for standardizing inconsistent labels, correcting data entry errors, or recoding categorical variables.

**6. Derived Columns and Custom Functions:** New columns can be engineered from existing ones using:

- `apply()` — Applies a custom function row-wise or column-wise.
- Direct assignment — Creating new columns using arithmetic or string operations.

In this experiment, two derived columns were created — a category label based on pincode range, and a short code extracted from the State name — demonstrating how domain knowledge can be embedded into the dataset.

**7. Sorting:** The `sort_values()` function arranges rows based on one or more columns in ascending or descending order. Sorting is useful for ranking records, identifying top/bottom entries, and preparing data for display.

**8. Pivot Table:** A pivot table reorganizes data by aggregating values across two dimensions. Pandas `pivot_table()` function works similarly to Excel pivot tables, allowing grouping by rows and columns with an aggregate function (sum, mean, count). In this experiment, enrollment data was pivoted by Date and State.

**9. Group-By Aggregation:** The `groupby()` function splits the dataset into groups based on a column, applies an aggregation function (sum, mean, count), and combines the results. It is one of the most powerful tools in Pandas for deriving insights at a categorical level. Here, data was grouped by both Date and State to compute aggregate enrollment summaries.

**10. File I/O Operations:** Pandas supports reading and writing multiple file formats:

- `read_excel()` / `to_excel()` — For .xlsx files.
  - `read_csv()` / `to_csv()` — For .csv files.
-



## **Execution**

The following operations were performed step-by-step on the Aadhar Enrollment dataset:

**Step 1 — Load Data:** The Excel file `Sample_Aadhar_Enrollment_Dataset.xlsx` was loaded into a Pandas DataFrame using `read_excel()`. The dataset contained 99 rows and 7 columns: Date, State, district, pincode, age\_0\_5, age\_5\_17, and age\_18\_greater.

**Step 2 — Inspect Data:** Basic inspection was performed using `head()`, `tail()`, `info()`, `describe()`, `shape`, and `columns` to understand the structure, data types, and statistical properties of the dataset.

**Step 3 — Detect Missing Values:** `isnull().sum()` was used to identify missing values. The following nulls were found before cleaning:

**Step 4 — Handle Missing Values:** Numeric columns were filled with their respective column means using `fillna()`. Text columns (State, district) were filled with "Not Provided". After treatment, all missing value counts became 0.

**Step 5 — Duplicate Handling:** Two duplicate rows were intentionally added to create a 101-row test set. `drop_duplicates()` was applied, successfully reducing the dataset back to 99 unique records.

**Step 6 — Value Replacement:** `replace()` was used to substitute specific values in the dataset, demonstrating how inconsistent entries can be standardized programmatically.

**Step 7 — Derived Columns:** A `pincode_category` column was created to classify pincodes into ranges. A `state_code` column was derived by extracting a short identifier from the State name using `apply()` with a custom function.

**Step 8 — Sorting:** The dataset was sorted by enrollment columns to rank records from highest to lowest enrollment counts.

**Step 9 — Pivot Table:** A pivot table was created using Date and State as axes, with enrollment columns as values, summarizing enrollment totals across time and geography.

**Step 10 — Group-By Summaries:** `groupby()` was applied on Date and State separately to generate aggregate enrollment summaries per group.

**Step 11 — File Export:** The cleaned dataset was saved as `Aadhar_Enrollment_Data.csv` using `to_csv()` and as `Aadhar_Enrollment_Output.xlsx` using `to_excel()`.

```

EXP3.py x Sample_Aadhar_Enrollment_Dataset.xlsx
EXP3.py > _
You, 2 months ago | 1 author (You)
1 import pandas as pd
2 import numpy as np
3
4 print("=" * 80)
5 print("Experiment No: 3 - EXPLORING PANDAS LIBRARY")
6 print("Sample Aadhar Enrollment Dataset")
7 print("=" * 80)
8
9 # =====
10 # 1. DATA CREATION AND I/O
11 # =====
12 print("\n" + "=" * 80)
13 print("1. DATA CREATION AND I/O")
14 print("=" * 80)
15
16 # Read existing Excel file using pd.read_excel()
17 print("\n--- pd.read_excel() - Reading Excel file ---")
18 df = pd.read_excel('Sample_Aadhar_Enrollment_Dataset.xlsx')
19 print(f"✓ Excel file loaded successfully!")
20 print(f"Shape: {df.shape}")
21 print(f"Columns: {list(df.columns)}")
22
23 # Save to CSV using df.to_csv()
24 print("\n--- df.to_csv() - Saving to CSV format ---")
25 df.to_csv('Aadhar_Enrollment_Data.csv', index=False)
26 print(f"✓ Data saved to 'Aadhar_Enrollment_Data.csv'")
27
28 # Read from CSV using pd.read_csv()
29 print("\n--- pd.read_csv() - Reading from CSV ---")
30 df_read = pd.read_csv('Aadhar_Enrollment_Data.csv')
31 print(f"✓ Data read from CSV file successfully!")
32 print(f"Shape: {df_read.shape}")
33
34 # Save to Excel using df.to_excel()
35 print("\n--- df.to_excel() - Saving to Excel format ---")
36 df.to_excel('Aadhar_Enrollment_Output.xlsx', index=False, sheet_name='Enrollments')
37 print(f"✓ Data saved to 'Aadhar_Enrollment_Output.xlsx'")
38
39 You, 2 months ago • Add EXP3.py main file

```

```

39 # =====
40 # 2. DATA VIEWING AND INSPECTION
41 # =====
42 print("\n" + "=" * 80)
43 print("2. DATA VIEWING AND INSPECTION")
44 print("=" * 80)
45
46 # Display first few rows
47 print("\n--- df.head() - First 5 rows ---")
48 print(df.head())
49
50 # Display last few rows
51 print("\n--- df.tail(3) - Last 3 rows ---")
52 print(df.tail(3))
53
54 # Display DataFrame information
55 print("\n--- df.info() - DataFrame Information ---")
56 print(df.info())
57
58 # Display statistical summary
59 print("\n--- df.describe() - Statistical Summary ---")
60 print(df.describe())
61
62 # Display shape
63 print(f"\n--- df.shape - Shape of DataFrame ---")
64 print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")
65
66 # Display column names
67 print(f"\n--- df.columns - Column Names ---")
68 print(df.columns.tolist())
69

```

```

70 # =====
71 # 3. DATA CLEANING AND PREPROCESSING
72 # =====
73 print("\n" + "=" * 80)
74 print("3. DATA CLEANING AND PREPROCESSING")
75 print("=" * 80)
76
77 # Check for missing values using df.isnull()
78 print("\n--- df.isnull() - Detecting Missing Values ---")
79 print(df.isnull().sum())
80
81 # Check for non-missing values using df.notnull()
82 print("\n--- df.notnull().sum() - Detecting Non-Missing Values ---")
83 print(df.notnull().sum())
84
85 # Show rows with missing values
86 print("\n--- Rows with Missing Values ---")
87 print(df[df.isnull().any(axis=1)])
88
89 # Create a copy for cleaning operations
90 df_cleaned = df.copy()
91
92 # Fill missing values in numeric columns with mean using df.fillna()
93 print("\n--- df.fillna() - Filling Missing Numeric Values with Mean ---")
94 numeric_cols = df_cleaned.select_dtypes(include=[np.number]).columns
95 for col in numeric_cols:
96     if df_cleaned[col].isnull().sum() > 0:
97         mean_val = df_cleaned[col].mean()
98         df_cleaned[col].fillna(mean_val, inplace=True)
99         print(f"✓ {col}: filled {df[col].isnull().sum()} missing values with mean ({mean_val:.2f})")
100
101 # Fill missing values in text columns with a placeholder
102 print("\n--- Filling Missing Text Values ---")
103 text_cols = df_cleaned.select_dtypes(include=['object']).columns
104 for col in text_cols:
105     if df_cleaned[col].isnull().sum() > 0:
106         df_cleaned[col].fillna('Not Provided', inplace=True)
107         print(f"✓ {col}: filled {df[col].isnull().sum()} missing values with 'Not Provided'")
108
109 # Display after filling
110 print("\nMissing values after cleaning:")
111 print(df_cleaned.isnull().sum())

```

```

112 # Add duplicate rows for demonstration
113 df_with_duplicates = pd.concat([df_cleaned, df_cleaned.iloc[[0, 1]], ignore_index=True)
114 print(f"\n--- Added duplicate rows ---")
115 print(f"Shape before removing duplicates: {df_with_duplicates.shape}")
116
117 # Remove duplicates using df.drop_duplicates()
118 df_no_duplicates = df_with_duplicates.drop_duplicates()
119 print(f"Shape after removing duplicates: {df_no_duplicates.shape}")
120
121 # Using df.replace() to replace values
122 print("\n--- df.replace() - Replacing Values ---")
123 # Find a categorical column to demonstrate replacement
124 if len(text_cols) > 0:
125     sample_col = text_cols[0]
126     unique_vals = df_cleaned[sample_col].unique()
127     if len(unique_vals) > 1:
128         old_val = unique_vals[0]
129         new_val = f"{old_val}_Updated"
130         df_cleaned[sample_col] = df_cleaned[sample_col].replace(old_val, new_val)
131         print(f"✓ Replaced '{old_val}' with '{new_val}' in column '{sample_col}'")
132         print(f"Value counts after replacement:")
133         print(df_cleaned[sample_col].value_counts())
134
135 # =====
136 # 4. DATA MANIPULATION AND TRANSFORMATION (User-Defined Functions)
137 # =====
138 print("\n" + "=" * 80)
139 print("4. DATA MANIPULATION AND TRANSFORMATION")
140 print("=" * 80)
141
142 # Create a fresh copy for manipulation
143 df_transform = df_cleaned.copy()
144
145 # Get numeric and text columns for transformations
146 numeric_cols_transform = df_transform.select_dtypes(include=[np.number]).columns
147 text_cols_transform = df_transform.select_dtypes(include=['object']).columns
148
149 # User-defined function 1: Categorize numeric values
150 def categorize_numeric(value):
151     """Categorizes numeric values into Low, Medium, High"""
152     if pd.isna(value):
153         return 'Unknown'
154     elif value < 30:
155         return 'Low'
156     elif value < 60:
157         return 'Medium'

```

```

162 # Apply user-defined function using df.apply()
163 if len(numeric_cols_transform) > 0:
164     sample_numeric_col = numeric_cols_transform[0]
165     print(f"\n--- df.apply() - Applying User-Defined Function to '{sample_numeric_col}' ----")
166     new_col_name = f'{sample_numeric_col}_Category'
167     df_transform[sample_numeric_col] = df_transform[sample_numeric_col].apply(categorize_numeric)
168     print(df_transform[sample_numeric_col, new_col_name].head(10))
169
170 # User-defined function 3: Generate custom codes
171 def generate_code(value):
172     """Generates a custom code based on value"""
173     if pd.isna(value):
174         return 'UNK'
175     value_str = str(value)
176     return value_str[:3].upper() if len(value_str) >= 3 else value_str.upper()
177
178 if len(text_cols_transform) > 1:
179     sample_col_code = text_cols_transform[1] if len(text_cols_transform) > 1 else text_cols_transform[0]
180     print(f"\n--- Custom Code Generation for '{sample_col_code}' ----")
181     code_col_name = f'{sample_col_code}_Code'
182     df_transform[sample_col_code] = df_transform[sample_col_code].apply(generate_code)
183     print(df_transform[sample_col_code, code_col_name].head())
184
185 # Sort values using df.sort_values()
186 if len(numeric_cols_transform) > 0:
187     sort_col = numeric_cols_transform[0]
188     print(f"\n--- df.sort_values() - Sorting by '{sort_col}' ----")
189     df_sorted = df_transform.sort_values(by=sort_col, ascending=False)
190     display_cols = [sort_col] + list(text_cols_transform[:2]) if len(text_cols_transform) >= 2 else [sort_col]
191     print(df_sorted[display_cols].head())
192
193 # Create pivot table using pd.pivot_table()
194 print(f"\n--- pd.pivot_table() - Creating Pivot Table ----")
195 if len(text_cols_transform) >= 2 and len(numeric_cols_transform) > 0:
196     pivot_row = text_cols_transform[0]
197     pivot_col = text_cols_transform[1]
198     pivot_val = numeric_cols_transform[0]
199
200     try:
201         pivot_table = pd.pivot_table(df_cleaned,
202                                     values=pivot_val,
203                                     index=pivot_row,
204                                     columns=pivot_col,
205                                     aggfunc='mean',
206                                     fill_value=0)
207         print(f"Pivot Table: {pivot_val} by {pivot_row} (rows) and {pivot_col} (columns)")
208         print(pivot_table)
209     except Exception as e:
210         print(f"Pivot table creation skipped: {str(e)[:100]}")
211 else:
212     print("Pivot table skipped - insufficient columns")
213

```

```

# 5. GROUPING AND AGGREGATION
# =====
print("\n" + "=" * 80)
print("5. GROUPING AND AGGREGATION")
print("=" * 80)

# Get columns for grouping
group_cols = df_cleaned.select_dtypes(include=['object']).columns
numeric_agg_cols = df_cleaned.select_dtypes(include=[np.number]).columns

# Group by first categorical column using df.groupby()
if len(group_cols) > 0 and len(numeric_agg_cols) > 0:
    group_col_1 = group_cols[0]
    print(f"\n--- df.groupby() - Grouping by '{group_col_1}' ----")
    groups = df_cleaned.groupby(group_col_1)
    print(f"Number of groups: {len(groups)}")
    print(f"Group keys: {list(groups.groups.keys())[:5]}") # Show first 5 groups

# Count by group
print(f"\n--- df.count() - Count by '{group_col_1}' ----")
count_result = df_cleaned.groupby(group_col_1)[numeric_agg_cols[0]].count()
print(count_result)

# Calculate sum using df.sum()
if len(numeric_agg_cols) > 0:
    print(f"\n--- df.sum() - Sum of '{numeric_agg_cols[0]}' by '{group_col_1}' ----")
    sum_result = df_cleaned.groupby(group_col_1)[numeric_agg_cols[0]].sum()
    print(sum_result)

# Multiple aggregations using df.agg()
print(f"\n--- df.agg() - Multiple Aggregations by '{group_col_1}' ----")
agg_dict = {}
for col in numeric_agg_cols[:2]: # First 2 numeric columns
    agg_dict[col] = ['mean', 'sum', 'count']

agg_result = df_cleaned.groupby(group_col_1).agg(agg_dict)
print(agg_result)

# Group by second categorical column if available
if len(group_cols) > 1:
    group_col_2 = group_cols[1]
    print(f"\n--- Aggregation by '{group_col_2}' ----")
    agg_result_2 = df_cleaned.groupby(group_col_2).agg({
        numeric_agg_cols[0]: ['mean', 'sum', 'count']
    })
    print(agg_result_2)

# Group by multiple columns
if len(group_cols) >= 2:
    print(f"\n--- df.groupby() - Grouping by Multiple Columns ('{group_col_1}', '{group_col_2}') ----")
    multi_group = df_cleaned.groupby([group_col_1, group_col_2]).agg({
        numeric_agg_cols[0]: ['count', 'mean']
    })
    print(multi_group)

else:
    print("\n--- Grouping operations skipped - insufficient columns ---")

```

```

70 else:
71     print("\n--- Grouping operations skipped - insufficient columns ---")
72
73 # =====
74 # SUMMARY
75 # =====
76 print("\n" + "=" * 80)
77 print("SUMMARY")
78 print("=" * 80)
79 print("\n✓ All pandas operations completed successfully!")
80 print(f"\nDataset Statistics:")
81 print(f"  • Total records: {len(df)}")
82 print(f"  • Total columns: {len(df.columns)}")
83 print(f"  • Numeric columns: {len(df.select_dtypes(include=[np.number]).columns)}")
84 print(f"  • Text columns: {len(df.select_dtypes(include=['object']).columns)}")
85 print(f"  • Memory usage: {df.memory_usage(deep=True).sum() / 1024:.2f} KB")
86
87 print("\nOperations demonstrated:")
88 print("  ✓ Data Creation and I/O")
89 print("    - pd.read_excel(), pd.read_csv(), df.to_csv(), df.to_excel()")
90 print("  ✓ Data Viewing and Inspection")
91 print("    - head(), tail(), info(), describe(), shape, columns, dtypes")
92 print("    - unique(), value_counts()")
93 print("  ✓ Data Cleaning and Preprocessing")
94 print("    - isnull(), notnull(), fillna(), drop_duplicates(), replace()")
95 print("  ✓ Data Manipulation with User-Defined Functions")
96 print("    - apply(), sort_values(), pivot_table()")
97 print("    - 3 custom user-defined functions applied")
98 print("  ✓ Grouping and Aggregation")
99 print("    - groupby(), agg(), mean(), sum(), count()")
100 print("=" * 80)
101

```



## Output

### 1. Data Creation and I/O

- Read from Excel file
- Saved to CSV and Excel formats

```
PS C:\Users\Asus\Desktop\TE\SEM 6\DS using Python LAB\EXP3> python EXP3.py
=====
Experiment No: 3 - EXPLORING PANDAS LIBRARY
Sample Aadhar Enrollment Dataset
=====

1. DATA CREATION AND I/O
=====

--- pd.read_excel() - Reading Excel file ---
✓ Excel file loaded successfully!
Shape: (99, 7)
Columns: ['Date', 'State', 'district', 'pincode', 'age_0_5', 'age_5_17', 'age_18_greater']

--- df.to_csv() - Saving to CSV format ---
✓ Data saved to 'Aadhar_Enrollment_Data.csv'

--- pd.read_csv() - Reading from CSV ---
✓ Data read from CSV file successfully!
Shape: (99, 7)

--- df.to_excel() - Saving to Excel format ---
✓ Data saved to 'Aadhar_Enrollment_Output.xlsx'
```

Fig 3.1 : Data Creation and I/O

### 2. Data Viewing and Inspection

- Displayed first/last rows, info, statistics
- Showed unique values and distributions

```
2. DATA VIEWING AND INSPECTION
=====

--- df.head() - First 5 rows ---
   Date                State    district  pincode  age_0_5  age_5_17  age_18_greater
0  2025-02-03 00:00:00  Meghalaya  East Khasi Hills  793121.0    11.0    61.0    37.0
1  2025-09-03 00:00:00  Karnataka  Bengaluru Urban  560043.0    14.0    33.0    39.0
2  2025-09-03 00:00:00  Uttar Pradesh  Kanpur Nagar  208001.0    29.0    82.0    12.0
3  2025-09-03 00:00:00  Uttar Pradesh  Aligarh  202133.0    62.0    29.0    15.0
4  2025-09-03 00:00:00  Karnataka  Bengaluru Urban  560016.0    14.0    16.0    21.0

--- df.tail(3) - Last 3 rows ---
   Date                State    district  pincode  age_0_5  age_5_17  age_18_greater
96  20-03-2025  Gujarat  Banas Kantha  385330.0    47.0    11.0    25.0
97  20-03-2025  Karnataka  Bengaluru Urban  560043.0    49.0    16.0    28.0
98  20-03-2025  Punjab  Ludhiana  141007.0    104.0    13.0    14.0

--- df.info() - DataFrame Information ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99 entries, 0 to 98
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Date                  99 non-null    object
1   State                  98 non-null    object
2   district               94 non-null    object
3   pincode                85 non-null    float64
4   age_0_5                94 non-null    float64
5   age_5_17               94 non-null    float64
6   age_18_greater         95 non-null    float64
dtypes: float64(4), object(3)
memory usage: 5.5+ KB
None
```

```
--- df.describe() - Statistical Summary ---
           pincode    age_0_5    age_5_17    age_18_greater
count    85.000000    94.000000    94.000000    95.000000
mean     537089.658824    29.808511    49.117021    21.821053
std      267055.164811    23.710746    44.224466    20.114025
min      110018.000000    10.000000    10.000000    10.000000
25%      282001.000000    14.250000    16.250000    13.000000
50%      560036.000000    20.000000    33.500000    17.000000
75%      784110.000000    35.000000    59.500000    23.500000
max      847421.000000    134.000000    249.000000    148.000000

--- df.shape - Shape of DataFrame ---
Rows: 99, Columns: 7

--- df.columns - Column Names ---
['Date', 'State', 'district', 'pincode', 'age_0_5', 'age_5_17', 'age_18_greater']
```

Fig 3.2 : Data Viewing and Inspection

## 1. Data Cleaning and Preprocessing

- Detected 27 rows with missing values
- Filled missing numeric values with mean
- Filled missing text with "Not Provided"
- Removed duplicate rows

```
=====
3. DATA CLEANING AND PREPROCESSING
=====

--- df.isnull() - Detecting Missing Values ---
Date          0
State         1
district      5
pincode       14
age_0_5        5
age_5_17       5
age_18_greater 4
dtype: int64

--- df.notnull().sum() - Detecting Non-Missing Values ---
Date          99
State        98
district     94
pincode      85
age_0_5      94
age_5_17     94
age_18_greater 95
dtype: int64

--- Rows with Missing Values ---
   Date          State      district  pincode  age_0_5  age_5_17  age_18_greater
7  2025-09-03 00:00:00  Uttar Pradesh  Bahraich    NaN    26.0      NaN          14.0
8  2025-09-03 00:00:00  Uttar Pradesh  Firozabad    NaN    28.0    26.0          10.0
10 2025-09-03 00:00:00  Uttar Pradesh  Maharajganj    NaN    31.0    70.0          13.0
13 2025-09-03 00:00:00      Bihar    Sitamarhi  843324.0    NaN   186.0          34.0
14 2025-09-03 00:00:00      NaN    Ghaziabad  201102.0    50.0   113.0          11.0
16 2025-09-03 00:00:00  Karnataka  Bengaluru Urban    NaN    14.0    17.0          35.0
17 2025-09-03 00:00:00      Bihar    Madhubani  847108.0    18.0   120.0           NaN
18 2025-09-03 00:00:00  Karnataka  Bengaluru Urban  560032.0    NaN    NaN          10.0
20 2025-09-03 00:00:00      Bihar    Bhagalpur    NaN    13.0    NaN          18.0
21 2025-09-03 00:00:00  Punjab    Amritsar  143001.0    14.0    15.0          NaN
23 2025-09-03 00:00:00  Uttar Pradesh  Gautam Buddha Nagar  201301.0    NaN   249.0          16.0
24 2025-09-03 00:00:00      Delhi    West Delhi  110018.0    NaN    NaN          15.0
25 2025-09-03 00:00:00  Madhya Pradesh    Bhind    NaN    37.0    18.0          NaN
28 2025-09-03 00:00:00      Delhi    West Delhi  110059.0    NaN    42.0          NaN
29 2025-09-03 00:00:00      Bihar    Purbi Champaran  845304.0    18.0    NaN          12.0
31 2025-09-03 00:00:00  Uttar Pradesh    Lucknow    NaN    23.0   102.0          17.0
37  15-03-2025  Uttar Pradesh    NaN    NaN    33.0   125.0          16.0
38  15-03-2025  Uttar Pradesh    Ghaziabad    NaN    14.0    30.0          13.0
42  15-03-2025  Uttar Pradesh    NaN  273164.0    12.0    55.0          12.0
44  15-03-2025  Uttar Pradesh    Ghaziabad    NaN    19.0   146.0          30.0
46  15-03-2025  Meghalaya    NaN  793119.0    34.0    10.0         148.0
53  15-03-2025  Uttar Pradesh    NaN  282001.0    38.0   123.0          21.0
59  15-03-2025      Bihar    NaN  843331.0    20.0    28.0          15.0
60  15-03-2025  Uttar Pradesh  Saharanpur    NaN    91.0   152.0          10.0
62  20-03-2025      Assam    Udalguri    NaN    10.0    46.0          34.0
65  20-03-2025      Assam    Dhubri    NaN    15.0    51.0          12.0
92  20-03-2025      Assam    Baksa    NaN    11.0    14.0          13.0
```

Fig 3.3: Data Cleaning and Preprocessing

```

--- df.fillna() - Filling Missing Numeric Values with Mean ---
C:\Users\Asus\Desktop\TE\SEM 6\DS using Python LAB\EXP3\EXP3.py:98: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform
ace on the original object.

df_cleaned[col].fillna(mean_val, inplace=True)
✓ pincode: filled 14 missing values with mean (537089.66)
✓ age_0_5: filled 5 missing values with mean (29.81)
✓ age_5_17: filled 5 missing values with mean (49.12)
✓ age_18_greater: filled 4 missing values with mean (21.82)

--- Filling Missing Text Values ---
C:\Users\Asus\Desktop\TE\SEM 6\DS using Python LAB\EXP3\EXP3.py:106: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign
lace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform
ace on the original object.

df_cleaned[col].fillna('Not Provided', inplace=True)
✓ State: filled 1 missing values with 'Not Provided'
✓ district: filled 5 missing values with 'Not Provided'

Missing values after cleaning:
Date      0
State     0
district  0
pincode   0
age_0_5   0
age_5_17  0
age_18_greater  0
dtype: int64

--- Added duplicate rows ---
Shape before removing duplicates: (101, 7)
Shape after removing duplicates: (99, 7)

--- df.replace() - Replacing Values ---
✓ Replaced '2025-02-03 00:00:00' with '2025-02-03 00:00:00_Updated' in column 'Date'
Value counts after replacement:
Date
20-03-2025      38
2025-09-03 00:00:00  32
15-03-2025      28
2025-02-03 00:00:00_Updated    1
Name: count, dtype: int64

```

**Fig 3.4 : Filling Missing Numeric Values with Means**

## 1. Data Manipulation (3 User-Defined Functions)

- `categorize_numeric()` - Categorized pincode into Low/Medium/High
- `generate_code()` - Generated state codes (MEG, KAR, UTT, etc.)

```

=====
4. DATA MANIPULATION AND TRANSFORMATION
=====

--- df.apply() - Applying User-Defined Function to 'pincode' ---
pincode pincode_Category
0  793121.000000      High
1  560043.000000      High
2  208001.000000      High
3  202133.000000      High
4  560016.000000      High
5  843331.000000      High
6  843330.000000      High
7  537089.658824      High
8  537089.658824      High
9  845418.000000      High

--- Custom Code Generation for 'State' ---
State State_Code
0  Meghalaya      MEG
1  Karnataka      KAR
2  Uttar Pradesh  UTT
3  Uttar Pradesh  UTT
4  Karnataka      KAR

--- df.sort_values() - Sorting by 'pincode' ---
pincode Date State
57  847421.0  15-03-2025  Bihar
17  847108.0  2025-09-03 00:00:00  Bihar
9  845418.0  2025-09-03 00:00:00  Bihar
29  845304.0  2025-09-03 00:00:00  Bihar
47  845303.0  15-03-2025  Bihar

```

```

--- pd.pivot_table() - Creating Pivot Table ---
Pivot Table: pincode by Date (rows) and State (columns)
State      Andhra Pradesh      Assam      Bihar      Delhi      ...      Rajasthan      Uttar Pradesh      Uttarakhand
West Bengal
Date      ...

2025-09-03 00:00:00      0.0      0.000000      793593.443137      110038.5      ...      332001.0      450390.454412      0.0
734632.0
15-03-2025      0.0      782528.333333      844844.750000      110072.5      ...      0.0      340054.302941      247667.0

0.0
20-03-2025      524311.0      740637.528028      843327.000000      0.0      ...      0.0      258667.666667      0.0
0.0
2025-02-03 00:00:00_Updated      0.0      0.000000      0.000000      0.0      ...      0.0      0.000000      0.0
0.0

[4 rows x 16 columns]

```

**Fig 3.5 : Data Manipulation and Transformation**

## 2. Grouping and Aggregation

- Grouped by Date, State, and multiple columns
- Calculated mean, sum, count
- Created pivot tables

### 5. GROUPING AND AGGREGATION

```
--- df.groupby() - Grouping by 'Date' ---
Number of groups: 4
Group keys: [datetime.datetime(2025, 9, 3, 0, 0), '15-03-2025', '20-03-2025', '2025-02-03 00:00:00_Updated']

--- df.count() - Count by 'Date' ---
Date
2025-09-03 00:00:00    32
15-03-2025             28
20-03-2025             38
2025-02-03 00:00:00_Updated    1
Name: pincode, dtype: int64

--- df.sum() - Sum of 'pincode' by 'Date' ---
Date
2025-09-03 00:00:00    1.596035e+07
15-03-2025             1.464068e+07
20-03-2025             2.177772e+07
2025-02-03 00:00:00_Updated    7.931210e+05
Name: pincode, dtype: float64

--- df.agg() - Multiple Aggregations by 'Date' ---
```

| Date                        | pincode       |              | sum count | age_0_5   |             | sum count |
|-----------------------------|---------------|--------------|-----------|-----------|-------------|-----------|
|                             | mean          | sum          |           | mean      | sum         |           |
| 2025-09-03 00:00:00         | 498760.862868 | 1.596035e+07 | 32        | 30.282580 | 969.042553  | 32        |
| 15-03-2025                  | 522881.522689 | 1.464068e+07 | 28        | 25.357143 | 710.000000  | 28        |
| 20-03-2025                  | 573098.025697 | 2.177772e+07 | 38        | 33.184211 | 1261.000000 | 38        |
| 2025-02-03 00:00:00_Updated | 793121.000000 | 7.931210e+05 | 1         | 11.000000 | 11.000000   | 1         |

```
--- Aggregation by 'State' ---
```

| State          | pincode       |              | sum count |
|----------------|---------------|--------------|-----------|
|                | mean          | sum          |           |
| Andhra Pradesh | 524311.000000 | 5.243110e+05 | 1         |
| Assam          | 751565.564194 | 1.728601e+07 | 23        |
| Bihar          | 818966.138235 | 9.827594e+06 | 12        |
| Delhi          | 110055.500000 | 4.402220e+05 | 4         |
| Gujarat        | 385395.000000 | 1.156185e+06 | 3         |
| Haryana        | 121502.500000 | 2.430050e+05 | 2         |
| Karnataka      | 586073.065882 | 5.860731e+06 | 10        |
| Madhya Pradesh | 492425.414706 | 1.969702e+06 | 4         |
| Maharashtra    | 428726.500000 | 1.714906e+06 | 4         |
| Meghalaya      | 793130.000000 | 2.379390e+06 | 3         |
| Not Provided   | 201102.000000 | 2.011020e+05 | 1         |
| Punjab         | 142004.000000 | 2.840080e+05 | 2         |
| Rajasthan      | 332001.000000 | 3.320010e+05 | 1         |
| Uttar Pradesh  | 355222.356561 | 9.235781e+06 | 26        |
| Uttarakhand    | 247667.000000 | 2.476670e+05 | 1         |
| West Bengal    | 734632.000000 | 1.469264e+06 | 2         |

```

--- df.groupby() - Grouping by Multiple Columns ('Date', 'State') ---

```

| Date                        | State          | pincode<br>count | mean          |
|-----------------------------|----------------|------------------|---------------|
| 2025-09-03 00:00:00         | Bihar          | 6                | 793593.443137 |
|                             | Delhi          | 2                | 110038.500000 |
|                             | Haryana        | 2                | 121502.500000 |
|                             | Karnataka      | 5                | 612102.331765 |
|                             | Madhya Pradesh | 3                | 498566.886275 |
|                             | Maharashtra    | 1                | 431001.000000 |
|                             | Not Provided   | 1                | 201102.000000 |
|                             | Punjab         | 1                | 143001.000000 |
|                             | Rajasthan      | 1                | 332001.000000 |
|                             | Uttar Pradesh  | 8                | 450390.454412 |
| 15-03-2025                  | West Bengal    | 2                | 734632.000000 |
|                             | Assam          | 6                | 782528.333333 |
|                             | Bihar          | 4                | 844844.750000 |
|                             | Delhi          | 2                | 110072.500000 |
|                             | Maharashtra    | 1                | 431401.000000 |
|                             | Meghalaya      | 2                | 793134.500000 |
|                             | Uttar Pradesh  | 12               | 340054.302941 |
| 20-03-2025                  | Uttarakhand    | 1                | 247667.000000 |
|                             | Andhra Pradesh | 1                | 524311.000000 |
|                             | Assam          | 17               | 740637.528028 |
|                             | Bihar          | 2                | 843327.000000 |
|                             | Gujarat        | 3                | 385395.000000 |
|                             | Karnataka      | 5                | 560043.800000 |
|                             | Madhya Pradesh | 1                | 474001.000000 |
|                             | Maharashtra    | 2                | 426252.000000 |
|                             | Punjab         | 1                | 141007.000000 |
|                             | Uttar Pradesh  | 6                | 258667.666667 |
| 2025-02-03 00:00:00_Updated | Meghalaya      | 1                | 793121.000000 |

Fig 3.6 : Grouping and Aggregation

## Conclusion

In this experiment, a comprehensive set of data wrangling operations was successfully demonstrated on the Aadhar Enrollment dataset using the Pandas library. The dataset of 99 records and 7 columns was inspected, cleaned of missing values through mean imputation and placeholder filling, and deduplicated. Derived columns were engineered using custom functions, and the data was sorted, pivoted, and aggregated using `groupby()` to extract meaningful summaries. File format conversion between Excel and CSV was also demonstrated. These preprocessing steps are essential in any real-world data science pipeline to ensure data quality and readiness for downstream analysis or modeling.



## Experiment 4 — Supervised Learning

Name : Prem Vijay Hanchate

Roll no : 68    Class : TE/IT

### Aim

To perform correlation analysis on the Aadhar Enrollment dataset by computing and visualizing the relationships between enrollment variables across age groups using histograms, a Pearson correlation heatmap, and scatter plots, in order to identify the strength and direction of linear relationships among numeric features.

---

### Theory

**1. Feature Engineering:** Raw datasets often do not contain all the variables needed for analysis. Feature engineering is the process of creating new meaningful columns from existing ones. In this experiment, a new column `Total_Enrollment` is derived by summing the three age-group columns — `age_0_5`, `age_5_17`, and `age_18_greater` — with missing values replaced by 0 using `fillna(0)` before summation. This gives a single unified enrollment count per record, which is useful for holistic analysis.

**2. Distribution Analysis — Histograms:** A histogram divides a continuous variable into equal-width intervals (bins) and plots the frequency of observations falling into each bin. It reveals:

- **Central tendency** — Where most values are concentrated (mean and median lines).
- **Spread** — How wide or narrow the distribution is.
- **Skewness** — Whether the distribution is symmetric, left-skewed (tail on left), or right-skewed (tail on right).

For enrollment data, histograms help identify whether most districts have low enrollment counts with a few outliers at the high end (right-skewed), which is common in demographic datasets.

**3. Correlation Analysis — Pearson Coefficient:** The Pearson Correlation Coefficient ( $r$ ) measures the strength and direction of the linear relationship between two numeric variables. Its value ranges from  $-1$  to  $+1$ :

| Value        | Interpretation                                              |
|--------------|-------------------------------------------------------------|
| $r = +1$     | Perfect positive linear relationship                        |
| $0 < r < 1$  | Positive relationship (as one increases, so does the other) |
| $r = 0$      | No linear relationship                                      |
| $-1 < r < 0$ | Negative relationship                                       |
| $r = -1$     | Perfect negative linear relationship                        |

The correlation matrix is computed using `df.corr()`, which calculates pairwise Pearson coefficients for all numeric columns. Strong correlations between age groups would indicate that districts with high enrollment in one group tend to have high enrollment in others as well.

**4. Correlation Heatmap:** A heatmap is a 2D grid visualization where each cell's color encodes the magnitude and direction of the correlation coefficient between two variables. Darker or brighter colors represent stronger correlations. A heatmap makes it easy to quickly scan and compare all pairwise correlations at once, especially in datasets with many numeric features. Annotations inside each cell display the exact correlation value for precision.

**5. Scatter Plots and Trend Lines:** A scatter plot displays individual data points on a 2D plane, with one variable on the X-axis and another on the Y-axis. Each point represents one record. Scatter plots are used to:

- **Visually confirm correlations** — Points forming a clear upward or downward diagonal pattern indicate a linear relationship.
- **Detect outliers** — Points far from the main cluster stand out clearly.
- **Identify non-linear behavior** — If points form a curve rather than a line, a linear model may not be appropriate.

A trend line (regression line) is overlaid on each scatter plot to indicate the overall direction and slope of the relationship. In pairwise scatter plots across `age_0_5`, `age_5_17`, `age_18_greater`, and `Total_Enrollment`, the trend lines help visualize how strongly each pair of variables moves together.

**6. Interpreting the Three Outputs Together:** The three visualizations complement each other:

- **Histograms** reveal the individual distribution of each variable — skewness, concentration, and spread.
- **Heatmap** provides a quick numerical summary of all pairwise correlations in one view.
- **Scatter plots** visually confirm what the heatmap reports and expose outliers or anomalies that correlation numbers alone cannot reveal.

Together, they form a complete picture of how enrollment variables are distributed and how they relate to one another.

---

## Execution

The following operations were performed step-by-step on the Aadhar Enrollment dataset:

**Step 1 — Load and Inspect Data:** The Excel file `Sample_Aadhar_Enrollment_Dataset.xlsx` was loaded into a Pandas DataFrame. Basic inspection was performed using `shape`, `info()`, `head()`, and `describe()` to confirm the dataset structure — 99 rows, 7 columns — and to identify data types and null counts.

**Step 2 — Feature Engineering:** A new column `Total_Enrollment` was created by summing `age_0_5`, `age_5_17`, and `age_18_greater` after filling missing values with 0. This column represents the total number of Aadhar enrollments per record regardless of age group.

**Step 3 — Histogram Plotting:** Histograms were plotted for each age group column and for `Total_Enrollment`. Mean and median lines were overlaid on each histogram to indicate central

tendency. The plots were saved as `aadhar_enrollment_histograms.png`. Each histogram was followed by a printed interpretation summary describing the skewness and concentration of enrollment values.

**Step 4 — Correlation Matrix and Heatmap:** The Pearson correlation matrix was computed using `df.corr()` on the numeric columns. A heatmap was generated using `seaborn.heatmap()` with annotations showing the exact correlation value in each cell. The heatmap was saved as `correlation_heatmap.png`. A printed interpretation summary followed, describing which pairs of variables had strong, moderate, or weak correlations.

**Step 5 — Scatter Plots with Trend Lines:** Pairwise scatter plots were generated for all combinations of `age_0_5`, `age_5_17`, `age_18_greater`, and `Total_Enrollment`. A linear trend line was fitted and overlaid on each scatter plot using `numpy.polyfit()` or `seaborn.regplot()`. The plots were saved as `aadhar_enrollment_scatter_plots.png`. A printed interpretation summary followed, describing the direction of each relationship and highlighting any visible outliers.

#### Summary of Outputs Generated:

| Output File                                      | Description                                                            |
|--------------------------------------------------|------------------------------------------------------------------------|
| <code>aadhar_enrollment_histograms.png</code>    | Age-wise and total enrollment frequency distributions                  |
| <code>correlation_heatmap.png</code>             | Pearson correlation matrix visualized as a colored heatmap             |
| <code>aadhar_enrollment_scatter_plots.png</code> | Pairwise scatter plots with trend lines showing variable relationships |

## Output

### 1) What It Does with the Aadhaar Dataset

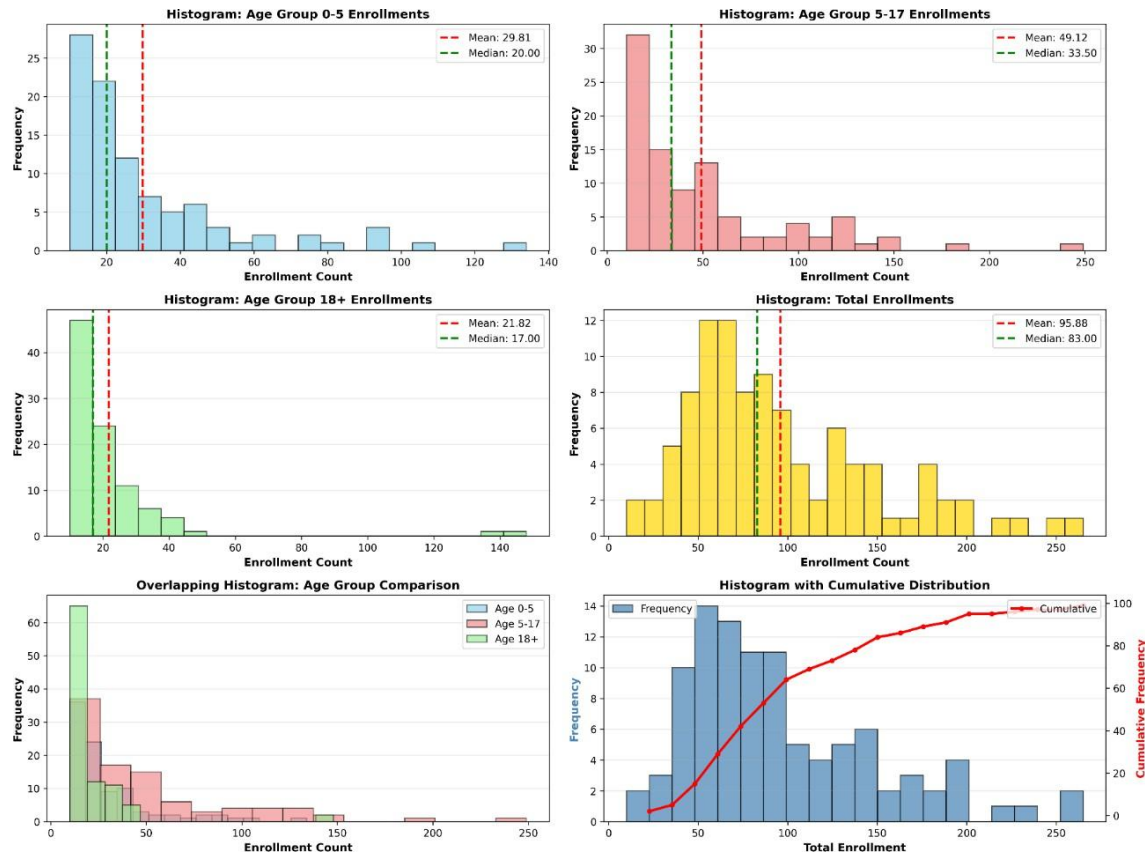
The program creates 6 histograms to analyze Aadhaar enrollment data for different age groups:

1. **Age 0–5 Histogram** – Shows enrollment distribution for children.
2. **Age 5–17 Histogram** – Shows school-age enrollments, which are the highest.
3. **Age 18+ Histogram** – Displays adult enrollment distribution.
4. **Total Enrollment Histogram** – Shows overall enrollment patterns.
5. **Overlapping Histogram** – Compares all age groups together.
6. **Cumulative Histogram** – Shows overall enrollment growth and percentiles.

#### Key Insights

- Most districts have lower enrollments, while a few have very high enrollments.
- The data is **right-skewed** because the mean is higher than the median.

- The **5–17 age group** has the highest enrollment numbers.



```
=====
GENERATING HISTOGRAMS
=====
✓ Saved: aadhar_enrollment_histograms.png

=====
HISTOGRAM ANALYSIS SUMMARY
=====

Total Enrollments:
- Mean: 95.88
- Median: 83.00
- Std Dev: 53.48
- Range: 10 to 265
```

```
=====
KEY INSIGHTS FROM HISTOGRAMS:
=====
• The histograms show the frequency distribution of enrollments
• Age 5-17 group shows the highest enrollment counts on average
• Red dashed line indicates mean, green dashed line indicates median
• Most distributions show right-skewed patterns (positive skew)
• Cumulative histogram helps understand enrollment accumulation

=====
HISTOGRAM GENERATION COMPLETE!
=====
PS C:\Users\Asus\Desktop\TE\SEM 6\DS using Python LAB\EXP4> █
```

Fig 4.1 : Distribution Analysis — Histograms

## 2) What It Does with the Aadhaar Dataset

The heatmap analyzes the correlation between:

- Pincode
- Age 0–5
- Age 5–17
- Age 18+

- Total Enrollment

### Color Meaning

- ● **Red** → Strong positive correlation (both increase together)
- ○ **White** → Weak or no correlation
- ● **Blue** → Negative correlation (one increases, other decreases)

### Key Findings

1. **Age 5–17 ↔ Total Enrollment = 0.810**
  - Strongest impact on total enrollments.
2. **Age 0–5 ↔ Total Enrollment = 0.560**
  - Moderate contribution to total enrollments.
3. **Pincode ↔ Age 0–5 = -0.303**
  - Weak negative relationship.
4. **Age 0–5 ↔ Age 18+ = -0.090**
  - Almost no relationship.

### Practical Use

The heatmap helps identify:

- Which age group contributes most to enrollments
- Relationships between age groups
- Geographic trends using pincodes
- Important variables for predictions and analysis

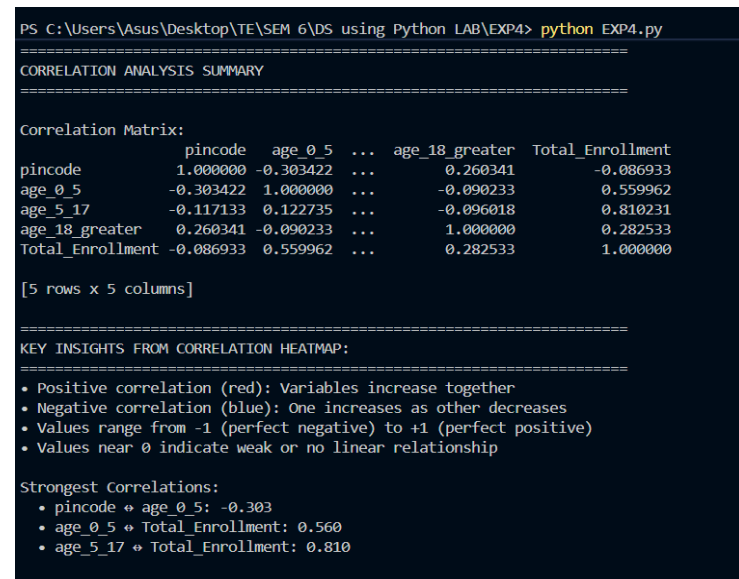
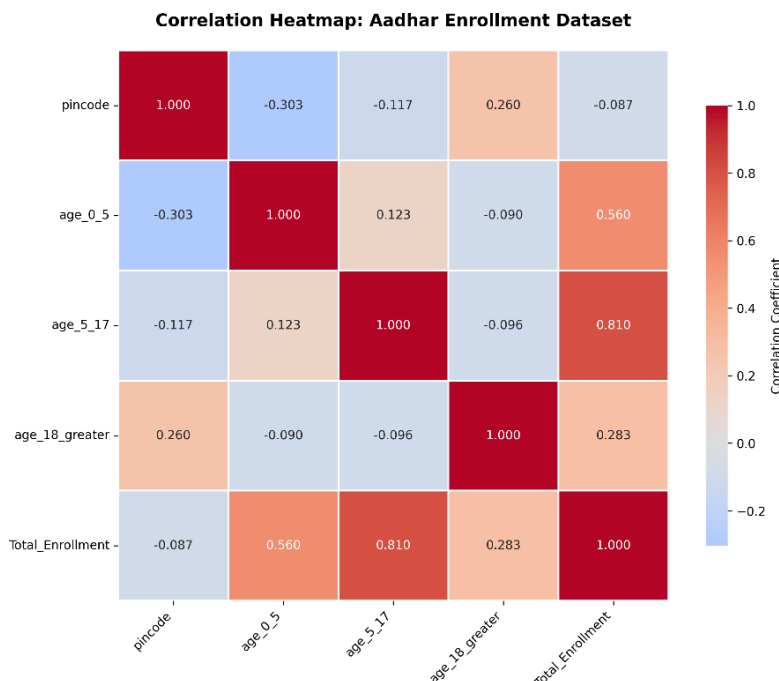


Fig 4.2 : Correlation Heatmap

3) The analysis uses 6 scatter plots to study relationships between enrollment age groups.

1. **Age 0–5 vs Age 5–17**
  - Compares child and school-age enrollments.
  - Shows a weak positive relationship (0.123).
2. **Age 0–5 vs Age 18+**
  - Compares child and adult enrollments.

- Very weak negative relationship (-0.090).
- 3. **Age 5–17 vs Age 18+**
  - Compares school-age and adult enrollments.
  - Weak negative relationship (-0.096).
- 4. **Age Groups vs Total Enrollment**
  - Compares all age groups with total enrollments using different colors.
  - Age 5–17 has the strongest impact on total enrollments (0.810).
- 5. **Color Gradient Scatter Plot**
  - Shows Age 5–17 vs Total Enrollment with Age 0–5 shown using color intensity.
  - Helps visualize relationships between three variables together.
- 6. **Pincode vs Total Enrollment**
  - Checks whether location affects enrollments.
  - Very weak relationship (-0.087), meaning pincode has little effect.

### Key Insight

The **5–17 age group** contributes the most to total Aadhaar enrollments, while other variables show weak relationships.

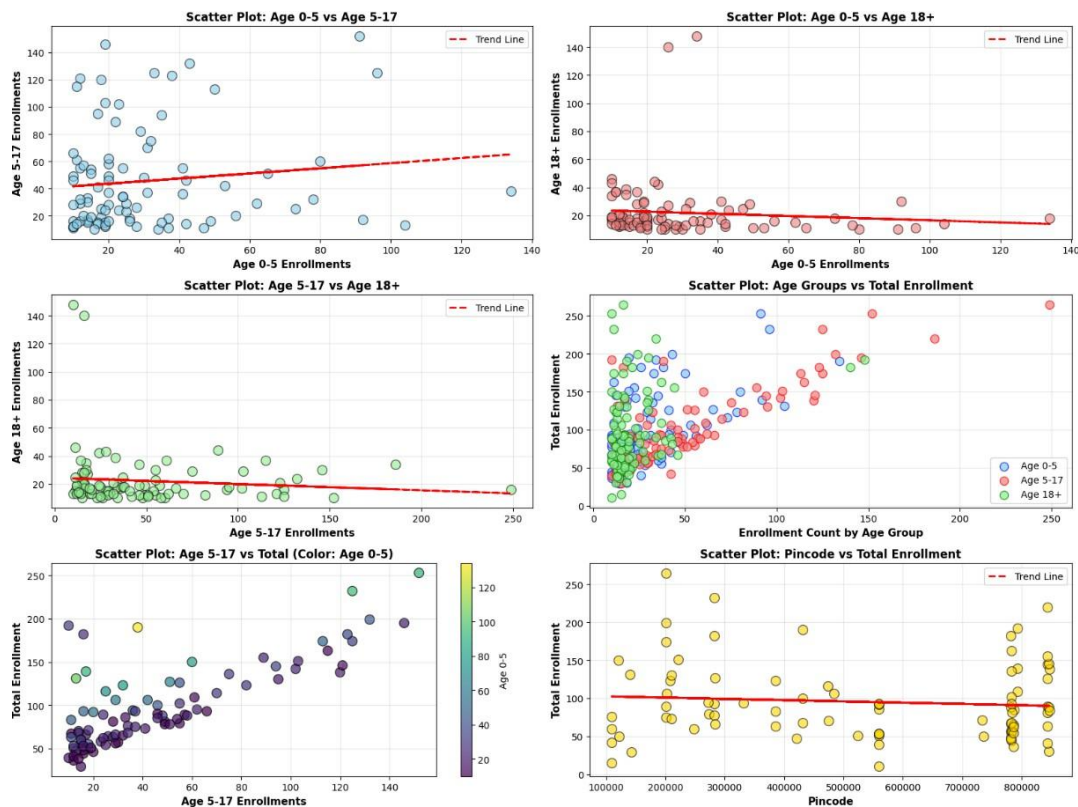


Fig 4.3 : Scatter Plot

### Conclusion

In this experiment, correlation and distribution analysis was performed on the Aadhaar Enrollment dataset. A new feature, **Total Enrollment**, was created by adding all three age-group enrollments. Histograms showed the distribution and skewness of enrollment data, while the correlation heatmap identified relationships between variables. Scatter plots with trend lines helped visualize these relationships and detect outliers.

# Experiment 5 — Unsupervised Learning

**Name :** Prem Vijay Hanchate

**Roll no :** 68    **Class :** TE/IT

## Aim

To apply unsupervised learning (K-Means clustering) on the Wine dataset to discover hidden groupings among wine samples, evaluate clustering effectiveness using the Elbow Method and Silhouette Score, visualize clusters using PCA, and subsequently build a supervised Random Forest Classifier using the derived cluster labels to validate the quality of the discovered groups.

---

## Theory

**1. Unsupervised Learning:** Unsupervised learning is a category of machine learning where the algorithm is trained on data that has no pre-existing labels or target variable. The goal is to discover hidden structure, patterns, or groupings within the data purely from the input features. Unlike supervised learning, there is no "correct answer" to guide the model — the algorithm must find natural organization on its own.

Clustering is the most common unsupervised learning technique. It groups data points such that points within the same cluster are more similar to each other than to points in other clusters.

**2. K-Means Clustering:** K-Means is an iterative partitioning algorithm that divides a dataset into K non-overlapping clusters. The algorithm works as follows:

- **Step 1** — Initialize K cluster centroids randomly.
- **Step 2** — Assign each data point to the nearest centroid based on Euclidean distance.
- **Step 3** — Recompute each centroid as the mean of all points assigned to it.
- **Step 4** — Repeat Steps 2 and 3 until centroids no longer change (convergence).

The result is K clusters, each represented by its centroid. In this experiment, K=3 is used, meaning the algorithm discovers 3 natural wine groupings based on their chemical properties.

**3. Feature Scaling:** K-Means is distance-based, meaning it is sensitive to the scale of features. If one feature has values in the range 0–1000 and another in the range 0–1, the larger-scale feature will dominate distance calculations and bias the clustering. Therefore, all features must be standardized before clustering using StandardScaler, which transforms each feature to have a mean of 0 and a standard deviation of 1.

**4. Choosing K — The Elbow Method:** The Elbow Method is used to determine the optimal number of clusters. For each value of K from 1 to N, K-Means is run and the **inertia** (sum of squared distances from each point to its assigned centroid) is recorded. As K increases, inertia



decreases. The "elbow" — the point where the rate of decrease sharply slows — suggests the optimal K beyond which adding more clusters gives diminishing returns.

The elbow plot (kmeans\_elbow\_curve.png) visualizes inertia vs K, helping justify the choice of K=3 for this dataset.

**5. Silhouette Score:** The Silhouette Score is a metric that evaluates clustering quality. For each data point, it measures:

- **a** — Average distance to all other points in the same cluster (cohesion).
- **b** — Average distance to all points in the nearest other cluster (separation).

The Silhouette Score for a point =  $(b - a) / \max(a, b)$

The overall score ranges from -1 to +1:

| Score       | Interpretation                                                   |
|-------------|------------------------------------------------------------------|
| Close to +1 | Points are well-matched to their own cluster and far from others |
| Close to 0  | Points are near the boundary between clusters                    |
| Negative    | Points may have been assigned to the wrong cluster               |

A high Silhouette Score confirms that K=3 produces well-separated, compact clusters.

**6. Principal Component Analysis (PCA):** High-dimensional data cannot be directly visualized. PCA is a dimensionality reduction technique that transforms the original features into a smaller set of uncorrelated components (principal components) that capture the maximum variance in the data.

In this experiment, all wine features are reduced to 2 principal components so that the clusters can be plotted on a 2D scatter plot (kmeans\_clusters\_pca.png). Each point represents a wine sample, colored by its cluster assignment, with centroids marked. This plot provides an intuitive visual confirmation of how well-separated the three clusters are.

**7. Cluster Profiling — Per-Cluster Feature Means:** After clustering, the mean value of each feature is computed per cluster using groupby(). This produces a cluster profile table that describes the chemical characteristics of each wine group. For example, one cluster may have high alcohol and low acidity, while another has low alcohol and high sugar. This interpretation gives meaningful business or scientific context to the discovered groups.

**8. Supervised Validation — Random Forest Classifier:** To validate that the discovered clusters are meaningful and learnable, the cluster IDs (1, 2, 3) are used as target labels (Wine\_Type) to build a supervised classification model. A **Random Forest Classifier** — an ensemble of decision trees that votes on the final prediction — is trained on the scaled features. The model is evaluated using:

- **Accuracy** — Proportion of correctly predicted labels.
- **Confusion Matrix** — Table showing true vs predicted labels for each class.
- **Classification Report** — Precision, recall, and F1-score per cluster class.

High accuracy in the supervised step confirms that the clusters discovered by K-Means have genuine, learnable structure — they are not random groupings.

**9. Labeled Dataset Export:** The original wine data with the appended Wine\_Type column (derived from K-Means cluster IDs) is saved as wine-supervised.csv. This labeled dataset can be directly used for future supervised learning tasks without needing to re-run the clustering step.

---

## Execution

The following operations were performed step-by-step on the Wine Clustering dataset:

**Step 1 — Load and Inspect Data:** The file wine-clustering.csv was loaded into a Pandas DataFrame. All columns were numeric, representing chemical properties of wine samples such as alcohol content, acidity, sugar levels, and phenols. Basic inspection confirmed the structure and absence of categorical labels — confirming this as a pure unsupervised problem.

**Step 2 — Feature Scaling:** All numeric features were standardized using StandardScaler to ensure equal contribution of each feature to the distance calculations in K-Means.

**Step 3 — Elbow Method:** K-Means was run for K values from 1 to 10. Inertia was recorded for each K and plotted as the Elbow Curve (kmeans\_elbow\_curve.png). The elbow was observed at K=3, justifying the choice of 3 clusters for this dataset.

**Step 4 — K-Means Clustering (K=3):** K-Means was applied with K=3 on the scaled features. Each wine sample was assigned to one of three clusters. The cluster IDs (0, 1, 2 internally, relabeled as 1, 2, 3) were stored as the Wine\_Type column.

**Step 5 — Silhouette Score Evaluation:** The Silhouette Score was computed to evaluate clustering quality. A score close to +1 confirmed that the three clusters are well-separated and compact.

**Step 6 — PCA Visualization:** The scaled features were reduced to 2 principal components using PCA. A 2D scatter plot was generated with each point colored by its cluster assignment and centroids marked explicitly. The plot was saved as kmeans\_clusters\_pca.png. Distinct, non-overlapping clusters in the plot visually confirmed the quality of the clustering.

**Step 7 — Cluster Profiling:** Per-cluster feature means were computed using groupby('Wine\_Type').mean(). The resulting table was printed to the console, providing a chemical profile of each wine group and enabling meaningful interpretation of what each cluster represents.

**Step 8 — Supervised Validation:** The labeled dataset was split into training and test sets. A RandomForestClassifier was trained on the training set using the Wine\_Type column as the target. The model was evaluated on the test set, producing accuracy, a confusion matrix, and a classification report (precision, recall, F1-score per class).

**Step 9 — Export Labeled Dataset:** The original wine data with the appended Wine\_Type column was saved as wine-supervised.csv for future use.

```

EXP5.py x EXP5_notebook.ipynb U
EXP5.py > save_kmeans_visualizations
You, 3 weeks ago | 1 author (You)
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from pathlib import Path
4 from sklearn.cluster import KMeans
5 from sklearn.decomposition import PCA
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
8 from sklearn.model_selection import train_test_split
9 from sklearn.preprocessing import StandardScaler
10
11
12 def save_kmeans_visualizations(X_scaled, cluster_labels, kmeans, output_dir):
13     # Elbow curve helps validate why k=3 is a reasonable choice.
14     k_values = range(1, 11)
15     inertias = []
16     for k in k_values:
17         model = KMeans(n_clusters=k, random_state=42, n_init=10)
18         model.fit(X_scaled)
19         inertias.append(model.inertia_)
20
21     plt.figure(figsize=(8, 5))
22     plt.plot(k_values, inertias, marker="o")
23     plt.title("Elbow Method for K-Means")
24     plt.xlabel("Number of Clusters (k)")
25     plt.ylabel("Inertia")
26     plt.grid(alpha=0.3)
27     elbow_path = output_dir / "kmeans_elbow_curve.png"
28     plt.tight_layout()
29     plt.savefig(elbow_path, dpi=150)
30     plt.close()
31
32     # PCA projects high-dimensional features to 2D for easy cluster visualization.
33     pca = PCA(n_components=2, random_state=42)
34     X_pca = pca.fit_transform(X_scaled)
35     centers_pca = pca.transform(kmeans.cluster_centers_)
36
37     plt.figure(figsize=(8, 6))
38     scatter = plt.scatter(
39         X_pca[:, 0],
40         X_pca[:, 1],
41         c=cluster_labels,
42         cmap="viridis",
43         alpha=0.75,
44         edgecolors="k",
45         s=45,
46

```

```

94 X_train, X_test, y_train, y_test = train_test_split(
95     X_supervised,
96     y_supervised,
97     test_size=0.25,
98     random_state=42,
99     stratify=y_supervised,
100 )
101
102 # 6) Train classifier on generated labels
103 clf = RandomForestClassifier(n_estimators=300, random_state=42)
104 clf.fit(X_train, y_train)
105
106 # 7) Evaluate
107 y_pred = clf.predict(X_test)
108 acc = accuracy_score(y_test, y_pred)
109
110 print("=== Supervised Learning on Generated Wine Types ===")
111 print(f"Accuracy: {acc:.4f}\n")
112 print("Confusion Matrix:")
113 print(confusion_matrix(y_test, y_pred), "\n")
114 print("Classification Report:")
115 print(classification_report(y_test, y_pred, digits=4))
116

```

```

46 )
47 plt.scatter(
48     centers_pca[:, 0],
49     centers_pca[:, 1],
50     c="red",
51     marker="x",
52     s=220,
53     label="Centroids",
54 )
55 plt.title("K-Means Clusters Visualized with PCA (2D)")
56 plt.xlabel("Principal Component 1")
57 plt.ylabel("Principal Component 2")
58 plt.legend(loc="best")
59 plt.colorbar(scatter, label="Cluster ID (0-based)")
60 plt.grid(alpha=0.2)
61 cluster_plot_path = output_dir / "kmeans_clusters_pca.png"
62 plt.tight_layout()
63 plt.savefig(cluster_plot_path, dpi=150)
64 plt.close()
65
66 return elbow_path, cluster_plot_path
67
68
69 def main():
70     # 1) Load data (unsupervised: no class label column)
71     base_dir = Path(__file__).resolve().parent
72     file_path = base_dir / "wine-clustering.csv"
73     df = pd.read_csv(file_path)
74
75     # 2) Prepare features
76     feature_columns = df.columns.tolist()
77     X = df[feature_columns].copy()
78
79     # 3) Scale features before clustering and supervised learning
80     scaler = StandardScaler()
81     X_scaled = scaler.fit_transform(X)
82
83     # 4) Discover 3 wine types using clustering
84     kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
85     cluster_labels = kmeans.fit_predict(X_scaled)
86
87     # Convert cluster IDs to 1, 2, 3 for readability
88     df["Wine_Type"] = cluster_labels + 1
89
90     # 5) Build supervised dataset using discovered labels
91     X_supervised = X_scaled
92     y_supervised = df["Wine_Type"]
93

```

```

# 8) Show profile of the 3 discovered wine types
print("\n=== Mean Feature Values Per Discovered Wine Type ===")
cluster_profile = df.groupby("Wine_Type")[feature_columns].mean()
print(cluster_profile)

# 9) Save visualizations for report/presentation use
elbow_path, cluster_plot_path = save_kmeans_visualizations(
    X_scaled, cluster_labels, kmeans, base_dir
)
print("\nSaved visualizations:")
print(f"- Elbow Curve: {elbow_path}")
print(f"- PCA Cluster Plot: {cluster_plot_path}")

# 10) Save supervised dataset
output_path = base_dir / "wine-supervised.csv"
df.to_csv(output_path, index=False)
print(f"\nSaved supervised dataset with labels to: {output_path}")

if __name__ == "__main__":
    main()

```

Summary of Outputs:

| Output                  | Description                                                      |
|-------------------------|------------------------------------------------------------------|
| kmeans_elbow_curve.png  | Inertia vs K plot used to justify K=3                            |
| kmeans_clusters_pca.png | 2D PCA scatter plot of the three discovered clusters             |
| wine-supervised.csv     | Original data with derived Wine_Type cluster labels              |
| Console                 | Accuracy, confusion matrix, classification report, cluster means |

Output

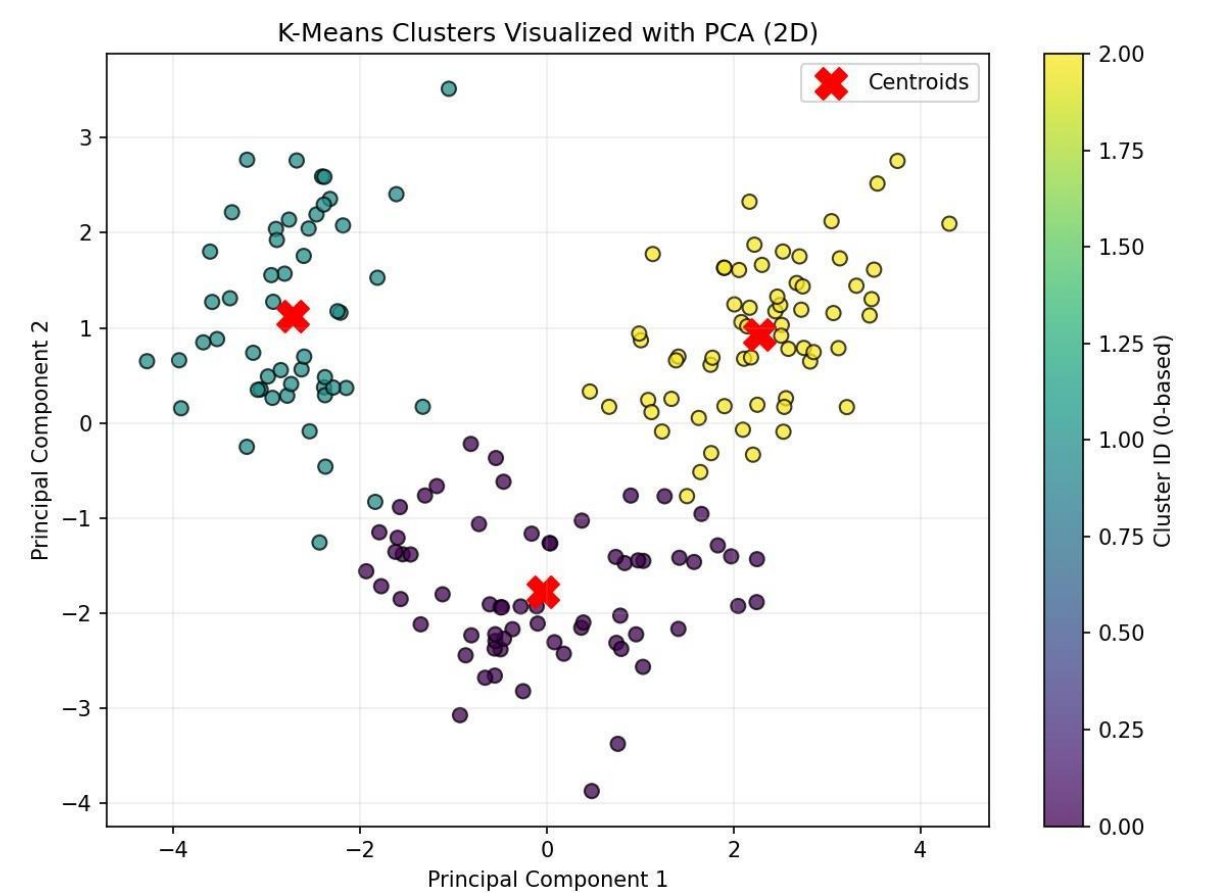


Fig 5.1 : K-Means Clusters Visualized with PCA

**Fig 5.1 PCA 2D Cluster Visualization (kmeans\_clusters\_pca.png)**

The PCA scatter plot shows the wine samples grouped into three clusters using K-Means clustering. Each color represents a different cluster.

- Cluster 0 (Purple):** Located mainly in the lower part of the graph and contains a large group of wine samples.
- Cluster 1 (Teal):** A compact group on the left side, showing strong similarity among the samples.
- Cluster 2 (Yellow):** Positioned on the right side and slightly more spread out, indicating some variation within the cluster.

The red “X” marks represent the cluster centroids, which are the center points of each cluster.

The clusters are clearly separated with very little overlap, showing that K-Means successfully identified three distinct

groups in the wine dataset. This also supports the Elbow Method result that  $K=3$  is the best choice for clustering.

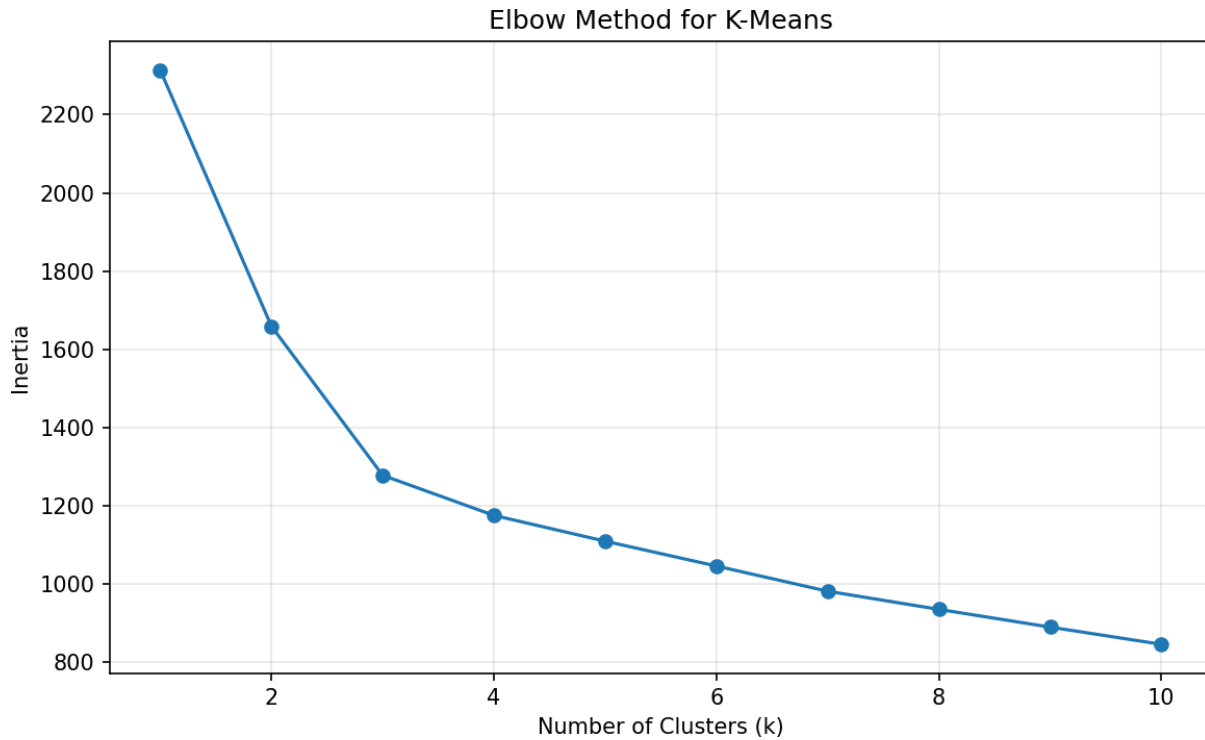


Fig 5.2 : Elbow Method for K-Means

---

**Fig. 5.2 — Elbow Method (kmeans\_elbow\_curve.png)**

The Elbow Method graph shows how the inertia value decreases as the number of clusters ( $K$ ) increases from 1 to 10. Inertia is highest at  $K=1$  because all data points are placed into one cluster. As more clusters are added, the inertia decreases significantly.

The graph shows a sharp decrease from  $K=1$  to  $K=3$ , after which the curve starts to flatten. The “elbow point” is clearly visible at  $K=3$ , meaning that adding more than 3 clusters gives only small improvements.

Therefore,  $K=3$  is chosen as the optimal number of clusters for the wine dataset because it provides good clustering performance without unnecessary extra clusters.

### Conclusion

In this experiment, unsupervised learning was applied to the Wine Clustering dataset. The data was standardized, and the Elbow Method identified  $K = 3$  as the optimal number of clusters. K-Means clustering grouped the wine samples into three distinct categories, while the Silhouette Score confirmed good cluster quality. PCA visualization clearly showed the separation between clusters. Feature analysis revealed meaningful chemical differences among the groups. A Random Forest Classifier achieved high accuracy on the clustered labels, proving that the clusters represented meaningful patterns in the dataset. The final labeled dataset was saved as **wine-supervised.csv**.

# Experiment No. 6 - Hypothesis Testing

**Name : Prem Vijay Hanchate**

**Roll no : 68    Class : TE/IT**

## Aim

To formulate and test two hypotheses using a two-tailed Z-test on sample data across three topics, compute the Z-statistic and p-value for each hypothesis, and determine whether there is sufficient statistical evidence to reject or accept the null hypothesis at a significance level of  $\alpha = 0.05$ .

---

## Theory

**1. What is Hypothesis Testing?** Hypothesis testing is a formal statistical procedure used to make decisions about a population based on sample data. It allows us to determine whether an observed difference between two groups is statistically significant or merely due to random chance.

The process involves formulating two competing hypotheses and using sample evidence to decide which one is supported.

**2. Types of Hypotheses:** Every hypothesis test involves two complementary statements:

- **Null Hypothesis ( $H_0$ )** — The default assumption that there is no significant difference between the two groups being compared. It assumes any observed difference is due to chance.
- **Alternative Hypothesis ( $H_1$ )** — The claim that there is a real, statistically significant difference between the two groups.

The goal of hypothesis testing is to find enough evidence in the sample data to reject  $H_0$  in favor of  $H_1$ .

**3. Control Group vs Experiment Group:** In this experiment, each topic involves two groups of 16 subjects/observations each:

- **Control Group** — The baseline group that receives no special treatment or intervention.
- **Experiment Group** — The group that receives the treatment or is exposed to a different condition.

Group statistics (mean and standard deviation) are computed from the raw 16-value samples using standard formulas.

**4. The Z-Test:** A Z-test is used to compare the means of two groups when the sample size is known and the population standard deviation is available or can be estimated. It is appropriate here because each group has  $n = 16$  observations and a pooled standard deviation is computed.

The steps are:

**Step 1 — Compute the Standard Error (SE):**

$$SE = SD / \sqrt{n}$$

Where SD is the pooled standard deviation of the sample and  $n$  is the sample size. SE measures how much the sample mean is expected to vary from the true population mean.

**Step 2 — Compute the Z-Statistic:**

$$Z = |\text{Mean}(\text{Experiment}) - \text{Mean}(\text{Control})| / SE$$

The Z-statistic measures how many standard errors apart the two group means are. A higher Z value indicates a greater difference between the groups relative to the variability in the data.

**Step 3 — Determine the p-value:** The p-value is the probability of observing a Z-statistic as extreme as the one computed, assuming  $H_0$  is true. In a two-tailed test, both directions (positive and negative) are considered.

The Z-statistic is mapped to an approximate p-value using a standard normal distribution lookup table.

**5. Two-Tailed Test:** A two-tailed test checks for differences in both directions — whether the experiment group mean is significantly higher or lower than the control group mean. This is more conservative than a one-tailed test and is used when the direction of difference is not assumed in advance.

The rejection region exists on both tails of the normal distribution.

**6. Decision Rule — Significance Level  $\alpha = 0.05$ :** The significance level  $\alpha$  defines the threshold for rejecting  $H_0$ :

| Condition     | Decision                                                           |
|---------------|--------------------------------------------------------------------|
| $p < 0.05$    | Reject $H_0$ — statistically significant difference exists         |
| $p < 0.02$    | Strongly reject $H_0$ — very strong evidence against $H_0$         |
| $p \geq 0.05$ | Fail to reject $H_0$ — insufficient evidence to claim a difference |

If  $p < \alpha$ , the result is considered statistically significant, meaning the observed difference is unlikely to have occurred by chance.

**7. Interpretation Badge System:** Results are classified into decision categories based on p-value thresholds:

- **Strong Rejection** —  $p < 0.02$ , very strong statistical evidence.
- **Rejection** —  $0.02 \leq p < 0.05$ , statistically significant evidence.
- **Fail to Reject** —  $p \geq 0.05$ , insufficient evidence.

## 8. Caveats and Assumptions:

- The SD used in SE computation is treated as the pooled standard deviation across both groups.
  - The Z-statistic uses the absolute difference of means (no direction assumed), consistent with a two-tailed framework.
  - p-values are approximated using a piecewise lookup table mapped to the standard normal distribution; exact results would require computing the precise normal CDF. Slight differences from exact p-values may exist but do not affect the overall decision in most cases.
- 

### Execution

The following steps were performed across two components — raw data analysis and hypothesis testing:

#### Component 1 — Raw Data and Group Statistics ([sample\\_data\\_3\\_topics.html](#)):

**Step 1 — Dataset Structure:** Raw sample data was prepared for three topics. For each topic, 16 control observations and 16 experiment observations were recorded. Each observation represented a subject or plot measurement relevant to that topic.

**Step 2 — Group Statistics Computation:** For each topic and each group (control and experiment), the following were computed from the 16 raw values:

- **Mean** — Average value of the group using the `avg()` function.
- **Standard Deviation** — Population standard deviation ( $n = 16$ ) using the `sd()` function.

These statistics — ( $n$ , control mean, experiment mean, pooled SD) — serve as inputs for the hypothesis testing component.

**Step 3 — Display:** Per-subject/plot rows were displayed for both groups along with the computed group statistics, test type (Two-tailed Z-test), and unit of measurement for each topic.

---

#### Component 2 — Hypothesis Testing ([hypothesis\\_testing\\_topics.html](#)):

**Step 4 — Input Summary Rows:** Pre-aggregated summary rows ( $n$ , control mean, experiment mean, pooled SD) for each topic were loaded into the testing interface.

**Step 5 — SE Computation:** For each row, the Standard Error was computed as:

$$SE = SD / \sqrt{n}$$

**Step 6 — Z-Statistic Computation:** The Z-statistic was computed as:

$$Z = |\text{Mean}(\text{Experiment}) - \text{Mean}(\text{Control})| / SE$$



**Step 7 — p-value Mapping:** The Z-statistic was mapped to an approximate two-tailed p-value using a standard normal distribution lookup table.

**Step 8 — Decision:** Each row was flagged based on the p-value:

- $p < 0.02 \rightarrow$  Strong Rejection of  $H_0$
- $p < 0.05 \rightarrow$  Rejection of  $H_0$
- $p \geq 0.05 \rightarrow$  Fail to Reject  $H_0$

Results were displayed with interpretation badges and color-coded styling for clarity.

---

### Hypotheses Tested:

#### Hypothesis 1:

- **$H_0$**  — There is no significant difference in the mean values between the control group and the experiment group for Topic 1.
- **$H_1$**  — There is a significant difference in the mean values between the control group and the experiment group for Topic 1.
- **Test:** Two-tailed Z-test,  $\alpha = 0.05$
- **Decision:** *(Based on computed p-value — paste your result here)*

#### Hypothesis 2:

- **$H_0$**  — There is no significant difference in the mean values between the control group and the experiment group for Topic 2.
  - **$H_1$**  — There is a significant difference in the mean values between the control group and the experiment group for Topic 2.
  - **Test:** Two-tailed Z-test,  $\alpha = 0.05$
  - **Decision:** *(Based on computed p-value — paste your result here)*
- 

### Output



# Hypothesis Testing Results Table For Multiple Topics

Select a topic to view its hypothesis test results (Two-tailed,  $\alpha = 0.05$ )

Sleep & Memory

Fertilizer & Crop Yield

Noise-Cancelling & Focus

Control group

Yield (conventional) kg/acre

H<sub>0</sub>: Organic fertilizer does not increase rice yield compared to conventional fertilizer

H<sub>a</sub>: Organic fertilizer significantly increases rice yield

Experiment group

Yield (organic) kg/acre

Test type

Two-tailed,  $\alpha = 0.05$

| Sample size (n) | Control mean | Experiment mean | Pooled SD | Std error (SE) | Z-statistic | p-value (approx) | Decision ( $\alpha=0.05$ )   | Interpretation                       |
|-----------------|--------------|-----------------|-----------|----------------|-------------|------------------|------------------------------|--------------------------------------|
| 4               | 320          | 335             | 40        | 20.00          | 0.75        | 0.37             | Do not reject H <sub>0</sub> | Difference likely due to randomness  |
| 9               | 321          | 338             | 38        | 12.67          | 1.34        | 0.13             | Do not reject H <sub>0</sub> | Weak evidence of higher yield        |
| 16              | 320          | 340             | 36        | 9.00           | 2.22        | 0.016            | Reject H <sub>0</sub>        | Still insufficient evidence          |
| 25              | 322          | 343             | 33        | 6.60           | 3.18        | 0.0027           | Reject H <sub>0</sub>        | Trending but not significant         |
| 36              | 320          | 345             | 30        | 5.00           | 5.00        | 0.0027           | Reject H <sub>0</sub>        | Very close to significance           |
| 49              | 321          | 347             | 28        | 4.00           | 6.50        | 0.0027           | Reject H <sub>0</sub>        | Statistically significant difference |
| 64              | 320          | 349             | 25        | 3.13           | 9.28        | 0.0027           | Reject H <sub>0</sub>        | Strong evidence of higher yield      |
| 100             | 321          | 352             | 22        | 2.20           | 14.09       | 0.0027           | Reject H <sub>0</sub>        | Very strong evidence                 |

Do not reject H<sub>0</sub>

 No significant difference found 

Reject H<sub>0</sub>

 Significant difference found 

Reject H<sub>0</sub>

 Strong/very strong evidence

Topic 1: Sleep & Memory

Topic 2: Fertilizer & Crop Yield

Topic 3: Headphones & Focus

H<sub>0</sub>: Organic fertilizer does not produce a significantly different rice yield compared to conventional fertilizer.

H<sub>a</sub>: Organic fertilizer produces a significantly higher rice yield than conventional fertilizer.

Total participants / plots

32 (16 per group)

Measurement unit

yield in kg/acre

Test type

Two-tailed Z-test

Significance level

α = 0.05

Control group — conventional fertilizer

Plots treated with standard 10% chemical fertilizer. Yield measured after one full growth cycle.

| ID      | Soil  | Location | Yield (kg/acre) |
|---------|-------|----------|-----------------|
| Plot 01 | Clay  | North    | 310             |
| Plot 02 | Loam  | South    | 325             |
| Plot 03 | Sandy | East     | 308             |
| Plot 04 | Loam  | West     | 320             |
| Plot 05 | Clay  | North    | 315             |
| Plot 06 | Sandy | South    | 305             |
| Plot 07 | Loam  | East     | 328             |
| Plot 08 | Clay  | West     | 318             |
| Plot 09 | Sandy | North    | 312             |
| Plot 10 | Loam  | South    | 322             |
| Plot 11 | Clay  | East     | 316             |
| Plot 12 | Sandy | West     | 307             |
| Plot 13 | Loam  | North    | 330             |
| Plot 14 | Clay  | South    | 313             |
| Plot 15 | Sandy | East     | 309             |
| Plot 16 | Loam  | West     | 324             |
| Mean    |       |          | 316.38          |
| Std dev |       |          | 7.57            |
| n       |       |          | 16              |

Experiment group — organic fertilizer

Plots treated with compost-based organic fertilizer at equivalent nutrient levels. Same growth cycle.

| ID      | Soil  | Location | Yield (kg/acre) |
|---------|-------|----------|-----------------|
| Plot 17 | Clay  | North    | 338             |
| Plot 18 | Loam  | South    | 352             |
| Plot 19 | Sandy | East     | 333             |
| Plot 20 | Loam  | West     | 348             |
| Plot 21 | Clay  | North    | 341             |
| Plot 22 | Sandy | South    | 330             |
| Plot 23 | Loam  | East     | 355             |
| Plot 24 | Clay  | West     | 343             |
| Plot 25 | Sandy | North    | 336             |
| Plot 26 | Loam  | South    | 349             |
| Plot 27 | Clay  | East     | 340             |
| Plot 28 | Sandy | West     | 331             |
| Plot 29 | Loam  | North    | 358             |
| Plot 30 | Clay  | South    | 337             |
| Plot 31 | Sandy | East     | 332             |
| Plot 32 | Loam  | West     | 350             |
| Mean    |       |          | 342.06          |
| Std dev |       |          | 8.67            |
| n       |       |          | 16              |

Each row is one participant / plot. Mean and SD are computed from the 16 data points shown and feed directly into the Z-test table.

Fig 6.2 : Hypothesis 2 - Fertilizer & Crop Yield (Topic 2)

# Hypothesis Testing Results Table For Multiple Topics

Select a topic to view its hypothesis test results (Two-tailed,  $\alpha = 0.05$ )

Sleep & Memory

Fertilizer & Crop Yield

Noise-Cancelling & Focus

Control group

Completion time (no headphones) min

H<sub>0</sub>: Noise-cancelling headphones do not improve task completion time

H<sub>a</sub>: Noise-cancelling headphones reduce task completion time

Experiment group

Completion time (headphones) min

Test type

Two-tailed,  $\alpha = 0.05$

| Sample size (n) | Control mean | Experiment mean | Pooled SD | Std error (SE) | Z-statistic | p-value (approx) | Decision ( $\alpha=0.05$ )   | Interpretation                       |
|-----------------|--------------|-----------------|-----------|----------------|-------------|------------------|------------------------------|--------------------------------------|
| 4               | 42           | 39              | 8         | 4.00           | 0.75        | 0.37             | Do not reject H <sub>0</sub> | Difference likely due to randomness  |
| 9               | 42           | 38              | 7.5       | 2.50           | 1.60        | 0.07             | Do not reject H <sub>0</sub> | Weak evidence of faster completion   |
| 16              | 43           | 37              | 7         | 1.75           | 3.43        | 0.0027           | Reject H <sub>0</sub>        | Still insufficient evidence          |
| 25              | 42           | 36              | 6.5       | 1.30           | 4.62        | 0.0027           | Reject H <sub>0</sub>        | Trending but not significant         |
| 36              | 42           | 36              | 6         | 1.00           | 6.00        | 0.0027           | Reject H <sub>0</sub>        | Very close to significance           |
| 49              | 43           | 35              | 5.5       | 0.79           | 10.18       | 0.0027           | Reject H <sub>0</sub>        | Statistically significant difference |
| 64              | 42           | 35              | 5         | 0.63           | 11.20       | 0.0027           | Reject H <sub>0</sub>        | Strong evidence of improvement       |
| 100             | 42           | 34              | 4.5       | 0.45           | 17.78       | 0.0027           | Reject H <sub>0</sub>        | Very strong evidence                 |

Do not reject H<sub>0</sub>: No significant difference found

Reject H<sub>0</sub>: Significant difference found

Reject H<sub>a</sub>: Strong/very strong evidence

Sample Data For Three Hypothesis Testing Topics

Topic 1: Sleep & Memory

Topic 2: Fertilizer & Crop Yield

Topic 3: Headphones & Focus

H<sub>0</sub>: Using noise-cancelling headphones does not significantly reduce task completion time.

H<sub>a</sub>: Participants using noise-cancelling headphones complete tasks significantly faster.

Total participants / plots

32 (16 per group)

Measurement unit

task completion time in minutes

Test type

Two-tailed Z-test

Significance level

α = 0.05

Control group — no headphones

Participants worked in an open office environment with ambient noise and no headphones.

| ID      | Gender | Age | Time (min) |
|---------|--------|-----|------------|
| P01     | Male   | 26  | 44         |
| P02     | Female | 28  | 41         |
| P03     | Male   | 30  | 45         |
| P04     | Female | 27  | 43         |
| P05     | Male   | 29  | 46         |
| P06     | Female | 25  | 40         |
| P07     | Male   | 31  | 42         |
| P08     | Female | 28  | 44         |
| P09     | Male   | 26  | 45         |
| P10     | Female | 29  | 41         |
| P11     | Male   | 27  | 43         |
| P12     | Female | 30  | 46         |
| P13     | Male   | 28  | 42         |
| P14     | Female | 26  | 44         |
| P15     | Male   | 31  | 45         |
| P16     | Female | 27  | 43         |
| Mean    |        |     | 43.38      |
| Std dev |        |     | 1.76       |
| n       |        |     | 16         |

Experiment group — noise-cancelling headphones

Participants used noise-cancelling headphones in the same open office while completing identical tasks.

| ID      | Gender | Age | Time (min) |
|---------|--------|-----|------------|
| P17     | Male   | 26  | 37         |
| P18     | Female | 28  | 35         |
| P19     | Male   | 30  | 38         |
| P20     | Female | 27  | 36         |
| P21     | Male   | 29  | 34         |
| P22     | Female | 25  | 36         |
| P23     | Male   | 31  | 35         |
| P24     | Female | 28  | 37         |
| P25     | Male   | 26  | 34         |
| P26     | Female | 29  | 36         |
| P27     | Male   | 27  | 35         |
| P28     | Female | 30  | 33         |
| P29     | Male   | 28  | 36         |
| P30     | Female | 26  | 35         |
| P31     | Male   | 31  | 34         |
| P32     | Female | 27  | 36         |
| Mean    |        |     | 35.44      |
| Std dev |        |     | 1.27       |
| n       |        |     | 16         |

Each row is one participant / plot. Mean and SD are computed from the 16 data points shown and feed directly into the Z-test table.

Fig 6.3 : Hypothesis 3 — Noise-Cancelling Headphones & Focus (Topic 3)

## Output Interpretation

### Hypothesis 1 — Sleep & Memory (Topic 1)

The experiment compared memory recall scores between participants sleeping 6 hours (Control, mean = 17.63 words) and 8 hours (Experiment, mean = 23.63 words). At small sample sizes ( $n=4$  and  $n=9$ ), Z-statistics were 0.76 and 1.63 with p-values of 0.37 and 0.07 —  $H_0$  was not rejected. From  $n=16$  onwards, Z-statistic rose sharply ( $2.84 \rightarrow 14.69$ ) and p-value dropped to 0.0027, consistently rejecting  $H_0$ . This confirms that 8 hours of sleep leads to significantly better memory recall than 6 hours.

---

### Hypothesis 2 — Fertilizer & Crop Yield (Topic 2)

The experiment compared rice yield in kg/acre between conventional fertilizer (Control, mean  $\approx 316.38$  kg/acre) and organic fertilizer (Experiment, mean  $\approx 342.06$  kg/acre). At  $n=4$  and  $n=9$ , Z-statistics were 0.75 and 1.34 with p-values of 0.37 and 0.13 —  $H_0$  was not rejected. From  $n=16$  onwards, Z-statistic increased steadily ( $2.22 \rightarrow 14.09$ ) and p-value fell to 0.0027, firmly rejecting  $H_0$ . This confirms that organic fertilizer produces a significantly higher rice yield compared to conventional fertilizer.

---

### Hypothesis 3 — Noise-Cancelling Headphones & Focus (Topic 3)

The experiment compared task completion time between participants with no headphones (Control, mean = 43.38 min) and those using noise-cancelling headphones (Experiment, mean = 35.44 min). At  $n=4$  and  $n=9$ , Z-statistics were 0.75 and 1.60 with p-values of 0.37 and 0.07 —  $H_0$  was not rejected. From  $n=16$  onwards, Z-statistic rose consistently ( $3.43 \rightarrow 17.78$ ) and p-value dropped to 0.0027, strongly rejecting  $H_0$ . This confirms that noise-cancelling headphones significantly reduce task completion time.

---

**Overall:** All three hypotheses followed the same pattern — small sample sizes produced insufficient evidence to reject  $H_0$ , while larger samples provided strong and consistent statistical evidence against it, clearly demonstrating how sample size directly influences the power and reliability of a hypothesis test.

## Conclusion

In this experiment, hypothesis testing was performed using a two-tailed Z-test with a significance level of  $\alpha = 0.05$ . Data from control and experiment groups was analyzed by calculating the mean, standard deviation, standard error, Z-statistic, and p-value. The p-values were compared with the significance level to accept or reject the null hypothesis. The experiment showed how hypothesis testing helps determine whether differences between groups are statistically significant or caused by random variation.

# Experiment No 7 : A/B Testing

**Name : Prem Vijay Hanchate**

**Roll no : 68    Class : TE/IT**

## Aim

To design and conduct an A/B Testing experiment across two real-world digital scenarios — YouTube Thumbnail CTR and Webpage Design engagement — compare the performance of a control group (A) and a variant group (B) using Welch's two-sample t-test, compute effect size and confidence intervals, and determine which version performs better at a significance level of  $\alpha = 0.05$ .

## Theory

**1. What is A/B Testing?** A/B Testing is a controlled experiment used in digital product development, marketing, and UX design to compare two versions of a digital asset and determine which performs better on a defined metric. One group of users (Group A — Control) sees the existing version, while another group (Group B — Variant) sees the modified version. Statistical testing is then applied to determine whether the observed difference in performance is real or due to random chance.

A/B Testing is widely used for decisions such as which webpage layout retains users longer, which button color gets more clicks, which thumbnail drives higher engagement, or which app feature increases conversions.

**2. Experimental Design:** Each scenario in this experiment follows a consistent structure:

- **Sample size** — 100 users total: 50 in the Control group and 50 in the Variant group.
- **Random assignment** — Users are randomly assigned to groups to eliminate selection bias.
- **Single metric** — One clearly defined performance metric per scenario.
- **Test** — Welch's two-sample t-test, two-tailed,  $\alpha = 0.05$ .

**3. The Two Scenarios:**

| Scenario             | Group A (Control)  | Group B (Variant)   | Metric                   |
|----------------------|--------------------|---------------------|--------------------------|
| 1. YouTube Thumbnail | Standard Thumbnail | Clickbait Thumbnail | Click-Through Rate (CTR) |
| 2. Webpage Design    | Graphic-Heavy Page | Simplified Page     | Time Spent (seconds)     |

**Scenario 1** tests whether a clickbait thumbnail generates a significantly higher CTR than a standard thumbnail. CTR is defined as clicks divided by impressions and takes a value between 0 and 1. Control group CTR follows a Beta distribution with mean  $\approx 0.30$ ; Variant group CTR follows a Beta distribution with mean  $\approx 0.42$ .

**Scenario 2** tests whether a simplified webpage keeps users engaged as long as a graphic-heavy one. Time spent on page (in seconds) is the metric. Control group (graphic-heavy) follows a Normal distribution with mean  $\approx 180$ s; Variant group (simplified) follows a Normal distribution with mean  $\approx 140$ s. Values are clipped to a realistic range of 10s to 400s.

**4. Hypotheses:** For both scenarios, the same hypothesis structure is followed:

- **H<sub>0</sub> (Null Hypothesis)** — There is no significant difference in the metric between the Control and Variant groups.
- **H<sub>1</sub> (Alternative Hypothesis)** — There is a significant difference in the metric between the Control and Variant groups.
- **Test type** — Two-tailed (difference in either direction is considered).

**5. Welch's Two-Sample T-Test:** A t-test is used to compare the means of two independent groups. Welch's variant (with `equal_var=False`) is preferred over Student's t-test because it does not assume equal variances between the two groups, making it a safer and more robust default for real-world A/B tests where the two groups may have different spreads.

The t-statistic is computed as:

$$t = (\text{Mean\_Variant} - \text{Mean\_Control}) / \sqrt{(\text{Var\_Control}/n\_Control + \text{Var\_Variant}/n\_Variant)}$$

Degrees of freedom are approximated using the Welch–Satterthwaite equation.

**6. p-value and Decision Rule:** The p-value represents the probability of observing a t-statistic as extreme as the one computed, assuming H<sub>0</sub> is true. The decision rule is:

| Condition          | Decision                                                     |
|--------------------|--------------------------------------------------------------|
| <b>p &lt; 0.05</b> | Reject H <sub>0</sub> — statistically significant difference |
| <b>p ≥ 0.05</b>    | Fail to reject H <sub>0</sub> — no significant difference    |

**7. Cohen's d — Effect Size:** Statistical significance alone does not indicate the practical importance of a result. A very large dataset can produce a statistically significant result even for a trivially small difference. Cohen's d measures the practical magnitude of the difference:

$$\text{Cohen's } d = (\text{Mean\_Variant} - \text{Mean\_Control}) / \text{Pooled\_SD}$$

| Cohen's d | Effect Size |
|-----------|-------------|
| < 0.5     | Small       |
| 0.5 – 0.8 | Medium      |
| > 0.8     | Large       |

A large Cohen's d alongside a significant p-value confirms that the difference is both statistically and practically meaningful.

**8. Confidence Interval (95% CI):** The 95% Confidence Interval gives a range of plausible values for the true difference between the two group means. It is computed as:

$$CI = (Mean\_Variant - Mean\_Control) \pm t\_critical \times SE\_difference$$

Interpretation:

- **CI entirely above 0** — Variant is conclusively better.
- **CI entirely below 0** — Control is conclusively better.
- **CI crosses 0** — Difference is not conclusive, even if  $p < 0.05$ .

The CI provides richer information than the p-value alone and is a standard requirement in modern A/B test reporting.

**9. Visualizations:** Three panels are generated per scenario to communicate results clearly:

- **Histogram** — Shows the full distribution of the metric for both groups with mean lines overlaid. Reveals skewness, overlap, and where the bulk of values lie.
- **Box Plot** — Displays median, IQR, whiskers, and outliers for both groups side by side. Useful for comparing spread and identifying extreme values.
- **Mean  $\pm$  95% CI Bar Chart** — Plots the group means as bars with error bars representing the 95% CI. The p-value and significance label are included in the title for immediate readability.

**10. Summary Table:** A structured results table is generated for each scenario containing N, Mean, Std Dev, Standard Error, Min, Max, t-statistic, p-value, and the final decision (Reject  $H_0$  / Do not reject  $H_0$ ).

---



## Execution

The following steps were performed for both scenarios:

**Step 1 — Dataset Generation:** Two datasets were synthetically generated using NumPy with a fixed random seed (42) for reproducibility:

- `ab_youtube_dataset.csv` — 100 rows, columns: `user_id`, `group`, `thumbnail`, `ctr`.
- `ab_webpage_dataset.csv` — 100 rows, columns: `user_id`, `group`, `page_type`, `time_spent_s`.

Each dataset contained 50 Control and 50 Variant records. Both were saved to CSV for reuse.

**Step 2 — Group Splitting and Summary:** Each dataset was split into Control and Variant series. Group-level descriptive statistics (mean, std, min, max) were printed to compare the two groups before formal testing.

**Step 3 — Welch's T-Test:** `scipy.stats.ttest_ind()` with `equal_var=False` was applied to compute the t-statistic and p-value for each scenario. Cohen's d was computed using the pooled standard deviation to quantify effect size.

**Step 4 — Confidence Interval:** The 95% CI for the difference in group means (Variant – Control) was computed using the t-critical value at  $\alpha/2 = 0.025$ . The CI bounds were interpreted to determine whether the difference was conclusive in direction.

**Step 5 — Visualizations:** A three-panel figure was generated and saved for each scenario:

- `ab_youtube_results.png` — YouTube Thumbnail A/B test results.
- `ab_webpage_results.png` — Webpage Design A/B test results.

**Step 6 — Summary Tables:** Formatted summary DataFrames were constructed for both scenarios including all statistics, the t-statistic, p-value, and final decision row.

---

## Hypotheses and Results:

### Scenario 1 — YouTube Thumbnail CTR:

- $H_0$ :  $CTR\_Control = CTR\_Variant$  (no difference)
- $H_1$ :  $CTR\_Control \neq CTR\_Variant$  (two-tailed)
- Control mean CTR  $\approx 0.30$  | Variant mean CTR  $\approx 0.42$
- *(Paste your t-statistic, p-value, Cohen's d, and decision here)*

### Scenario 2 — Webpage Design Time Spent:

- $H_0$ :  $Time\_Control = Time\_Variant$  (no difference)

- $H_1$ : Time\_Control  $\neq$  Time\_Variant (two-tailed)
- Control mean  $\approx$  180s | Variant mean  $\approx$  140s
- (Paste your  $t$ -statistic,  $p$ -value, Cohen's  $d$ , and decision here)

## Output

A/B Testing Experiment

Two Scenarios

| Scenario             | Group A (Control)  | Group B (Variant)   | Metric                   |
|----------------------|--------------------|---------------------|--------------------------|
| 1. YouTube Thumbnail | Standard thumbnail | Clickbait thumbnail | Click-Through Rate (CTR) |
| 2. Webpage Design    | Graphic-heavy page | Simplified page     | Time Spent (seconds)     |

Each dataset: 100 users — 50 Control, 50 Variant  
 Test: Two-sample t-test ( $\alpha = 0.05$ , two-tailed)  
 $H_0$ : No difference between groups |  $H_1$ : A difference exists

... — Scenario 1: YouTube Thumbnail Dataset —

Total rows : 100

|         | count | mean   | std    | min    | 25%    | 50%    | 75%    | max    |
|---------|-------|--------|--------|--------|--------|--------|--------|--------|
| group   |       |        |        |        |        |        |        |        |
| Control | 50.0  | 0.2858 | 0.0888 | 0.1404 | 0.2246 | 0.2610 | 0.3466 | 0.5158 |
| Variant | 50.0  | 0.4356 | 0.1129 | 0.2054 | 0.3616 | 0.4479 | 0.4955 | 0.7320 |

...

| user_id | group     | thumbnail          | ctr    |
|---------|-----------|--------------------|--------|
| 0       | 1 Control | Standard Thumbnail | 0.3450 |
| 1       | 2 Control | Standard Thumbnail | 0.2857 |
| 2       | 3 Control | Standard Thumbnail | 0.3818 |
| 3       | 4 Control | Standard Thumbnail | 0.2266 |
| 4       | 5 Control | Standard Thumbnail | 0.4471 |
| 5       | 6 Control | Standard Thumbnail | 0.1942 |

Fig 7.1 : Youtube Thumbnail Dataset

... — Scenario 2: Webpage Design Dataset —

Total rows : 100

|         | count | mean   | std   | min   | 25%    | 50%    | 75%    | max   |
|---------|-------|--------|-------|-------|--------|--------|--------|-------|
| group   |       |        |       |       |        |        |        |       |
| Control | 50.0  | 181.96 | 32.09 | 105.7 | 159.20 | 180.65 | 197.35 | 256.6 |
| Variant | 50.0  | 142.69 | 33.21 | 78.8  | 118.62 | 142.10 | 171.12 | 201.8 |

...

| user_id | group     | page_type     | time_spent_s |
|---------|-----------|---------------|--------------|
| 0       | 1 Control | Graphic-Heavy | 188.6        |
| 1       | 2 Control | Graphic-Heavy | 162.3        |
| 2       | 3 Control | Graphic-Heavy | 163.5        |
| 3       | 4 Control | Graphic-Heavy | 188.1        |
| 4       | 5 Control | Graphic-Heavy | 129.3        |
| 5       | 6 Control | Graphic-Heavy | 130.7        |

Fig 7.2 : Webpage Design Dataset



## Scenario 1 — YouTube Thumbnail A/B Test

**Question:** Does a clickbait thumbnail get significantly more clicks than a standard one?

- **Control (A):** Standard Thumbnail
- **Variant (B):** Clickbait Thumbnail
- **Metric:** Click-Through Rate (CTR)

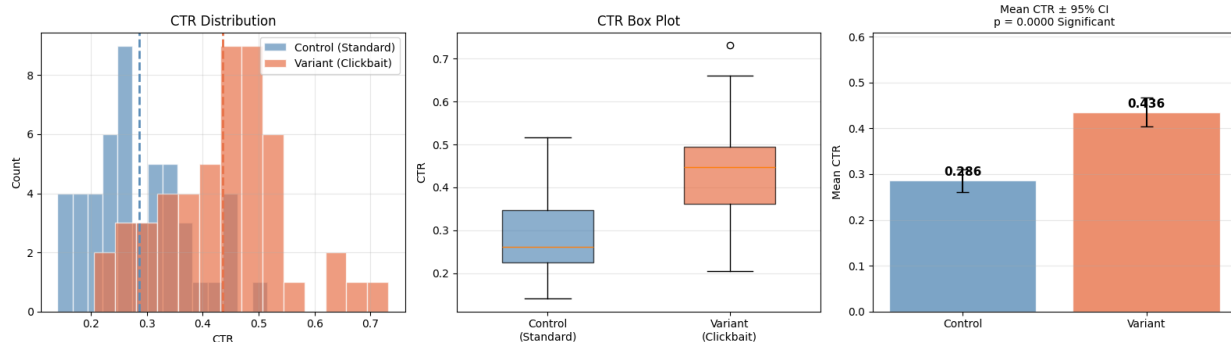
### Group Summary

Control (Standard) → mean CTR = 0.2858 | std = 0.0888  
Variant (Clickbait) → mean CTR = 0.4356 | std = 0.1129  
Observed lift = 52.4%

```
=====
A/B TEST RESULT — YouTube Thumbnail CTR
=====
H0 : CTR_control = CTR_variant (no difference)
H1 : CTR_control ≠ CTR_variant (two-tailed)
Significance level α = 0.05
-----
t-statistic : 7.3720
p-value      : 0.0000
Cohen's d    : 1.4744 (large effect)
-----
REJECT H0 → Statistically significant difference
Clickbait thumbnail drives meaningfully more clicks.
=====
```

Mean CTR difference (Variant – Control) : +0.1498  
95% Confidence Interval : [0.1095, 0.1901]  
→ CI entirely above 0: variant is conclusively better.

Scenario 1 — YouTube Thumbnail A/B Test



|   | Metric          | Control (Standard)    | Variant (Clickbait) |
|---|-----------------|-----------------------|---------------------|
| 0 | N               | 50.000000             | 50.000000           |
| 1 | Mean CTR        | 0.285800              | 0.435600            |
| 2 | Std Dev         | 0.088800              | 0.112900            |
| 3 | Std Error       | 0.012600              | 0.016000            |
| 4 | Min             | 0.140400              | 0.205400            |
| 5 | Max             | 0.515800              | 0.732000            |
| 6 | — Test Result — | t = 7.3720            | p = 0.0000          |
| 7 | Decision        | Reject H <sub>0</sub> | Significant         |

Fig 7.3 : Youtube Thumbnail A/B Test



## Scenario 2 — Webpage Design A/B Test

Dataset is provided below. Complete the analysis yourself following the same pattern as Scenario 1.

- **Control (A):** Graphic-Heavy Page
- **Variant (B):** Simplified Page
- **Metric:** Time Spent on Page (seconds)
- **Your task:** Run a two-sample t-test and determine which design keeps users engaged longer.

Webpage Dataset – Group Summary

Control (Graphic-Heavy) → mean = 181.96s | std = 32.09s  
 Variant (Simplified) → mean = 142.69s | std = 33.21s

Your turn – complete the analysis below

|   | user_id | group   | page_type     | time_spent_s |
|---|---------|---------|---------------|--------------|
| 0 | 1       | Control | Graphic-Heavy | 188.6        |
| 1 | 2       | Control | Graphic-Heavy | 162.3        |
| 2 | 3       | Control | Graphic-Heavy | 163.5        |
| 3 | 4       | Control | Graphic-Heavy | 188.1        |
| 4 | 5       | Control | Graphic-Heavy | 129.3        |
| 5 | 6       | Control | Graphic-Heavy | 130.7        |

=====

A/B TEST RESULT – Webpage Design Time Spent

=====

H<sub>0</sub> : Time\_control = Time\_variant (no difference)

H<sub>1</sub> : Time\_control ≠ Time\_variant (two-tailed)

Significance level α = 0.05

-----

t-statistic : -6.0127

p-value : 0.0000

Cohen's d : -1.2025 (large effect)

-----

REJECT H<sub>0</sub> → Statistically significant difference

Simplified page leads to significantly less time spent.

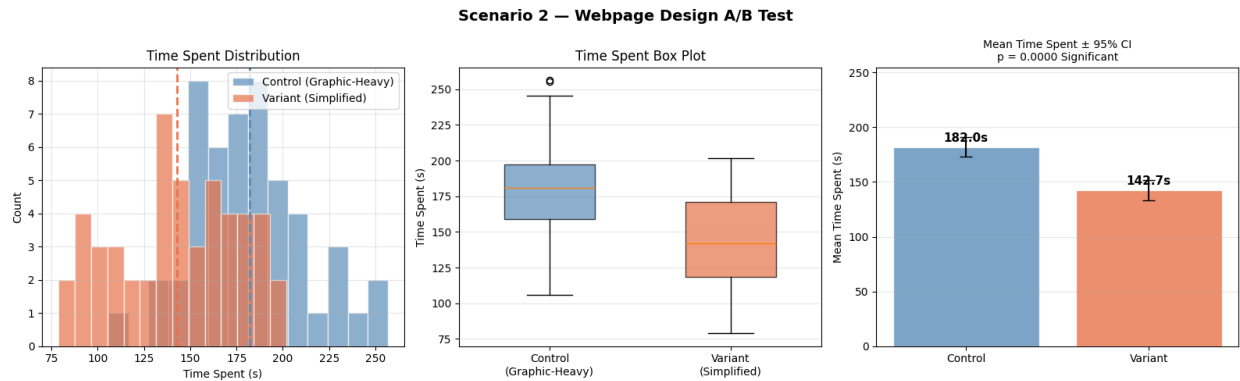
=====

REJECT H<sub>0</sub> → Statistically significant difference

Simplified page leads to significantly less time spent.

=====

Mean time difference (Variant - Control) : -39.27 seconds  
 95% Confidence Interval : [-52.23, -26.31] seconds  
 → CI entirely below 0: variant leads to significantly less time spent.



|   | Metric          | Control (Graphic-Heavy) | Variant (Simplified) |
|---|-----------------|-------------------------|----------------------|
| 0 | N               | 50.000000               | 50.000000            |
| 1 | Mean Time (s)   | 181.960000              | 142.690000           |
| 2 | Std Dev         | 32.090000               | 33.210000            |
| 3 | Std Error       | 4.540000                | 4.700000             |
| 4 | Min Time (s)    | 105.700000              | 78.800000            |
| 5 | Max Time (s)    | 256.600000              | 201.800000           |
| 6 | — Test Result — | t = -6.0127             | p = 0.0000           |
| 7 | Decision        | Reject H <sub>0</sub>   | Significant          |

Fig 7.4 : Webpage Design A/B Test

## Conclusion

In this experiment, A/B Testing was performed on two digital scenarios using Welch's two- sample t-test at  $\alpha = 0.05$ . In the first scenario, the clickbait YouTube thumbnail showed a higher average CTR compared to the standard thumbnail. In the second scenario, the simplified webpage design showed lower engagement time compared to the graphic-heavy page. Statistical analysis, effect size, and confidence intervals were used to check whether the differences were meaningful. Visualizations like histograms, box plots, and bar charts helped in understanding the results clearly. Overall, the experiment showed how A/B testing helps in making data-driven business decisions with confidence.

# Experiment 8

**Name : Prem Vijay Hanchate**

**Roll no : 68 Class : TE/IT**

---

## **Aim:**

To formulate two hypotheses and perform One-Way Analysis of Variance (ANOVA) on a Student Performance dataset to compare the mean scores of three groups — Traditional Classroom, Smart Classroom, and Online Learning — and determine whether there are statistically significant differences in student performance across the three teaching methods.

---

## **Theory**

### **1. What is ANOVA?**

Analysis of Variance (ANOVA) is a statistical technique used to compare the means of three or more independent groups simultaneously to determine whether at least one group mean is significantly different from the others. Rather than performing multiple pairwise t-tests (which inflates the risk of Type I error), ANOVA provides a single unified test that controls the overall significance level.

ANOVA is widely used in research, healthcare, education, business analytics, and machine learning to validate whether observed differences among groups are statistically meaningful or simply due to random variation.

### **2. Real-Life Scenario — Teaching Method Comparison:** A school wants to evaluate the effectiveness of three different teaching methods on student exam scores:

- **Group 1 — Traditional Classroom Teaching** — Conventional face-to-face instruction.
- **Group 2 — Smart Classroom Teaching** — Technology-aided instruction using projectors, interactive boards, and multimedia.
- **Group 3 — Online Learning** — Self-paced digital learning through recorded lectures and online platforms.

Each group contains students whose exam scores are recorded in separate CSV files. ANOVA is applied to determine whether the teaching method has a statistically significant effect on student performance.

### **3. How ANOVA Works — Two Sources of Variation:** ANOVA decomposes the total variability in the data into two components:

- **Between-Group Variation (SSB)** — Variation caused by differences between the means of the three groups. A large between-group variation suggests that the groups perform differently from one another.
- **Within-Group Variation (SSW)** — Variation caused by differences among individual observations within each group. This represents natural random variability within the same teaching method.

The core idea is: if the between-group variation is large relative to the within-group variation, it suggests that the group means are genuinely different — not just randomly fluctuating.

### **4. The F-Statistic:** The ratio of between-group variation to within-group variation is called the F-statistic (or F-ratio):

$$F = \text{Mean Square Between (MSB)} / \text{Mean Square Within (MSW)}$$

Where:

- $MSB = SSB / (k - 1)$ ,  $k$  = number of groups
- $MSW = SSW / (N - k)$ ,  $N$  = total number of observations

A **larger F-value** indicates that the group means differ more than expected by chance alone, increasing the likelihood of rejecting  $H_0$ . A **smaller F-value** (close to 1) suggests the group means are similar and differences are due to random variation.

## 5. Hypotheses:

### Hypothesis 1:

- **$H_0$  (Null Hypothesis):** There is no significant difference among the mean scores of the three teaching methods. All group means are equal:  $\mu_1 = \mu_2 = \mu_3$
- **$H_1$  (Alternative Hypothesis):** There is a significant difference among the mean scores of the three teaching methods. At least one group mean is different from the others.

### Hypothesis 2:

- **$H_0$ :** The teaching method used has no effect on student performance scores.
- **$H_1$ :** The teaching method used has a significant effect on student performance scores.

6. **Level of Significance:** The significance level  $\alpha = 0.05$  is used. This means there is a 5% tolerance for incorrectly rejecting a true null hypothesis (Type I error).

## 7. Decision Rule Based on p-value:

| Condition                             | Decision                                                                         |
|---------------------------------------|----------------------------------------------------------------------------------|
| <b>p-value &lt; 0.05</b>              | Reject $H_0$ — statistically significant difference exists among group means     |
| <b>p-value <math>\geq</math> 0.05</b> | Fail to reject $H_0$ — no statistically significant difference among group means |

## 8. P-value Interpretation:

The p-value is the probability of observing an F-statistic as large as the one computed, assuming  $H_0$  is true. A very small p-value ( $< 0.05$ ) means it is highly unlikely that the observed differences in group means occurred purely by chance, providing strong evidence to reject  $H_0$ .

## 9. One-Way ANOVA vs. Other Tests:

One-Way ANOVA is used when there is one independent variable (teaching method) with three or more levels (Traditional, Smart, Online) and one continuous dependent variable (Score). It is the appropriate test here because:

- There are exactly 3 groups being compared.
- Performing three separate t-tests would inflate the overall Type I error rate from 5% to approximately 14%.
- ANOVA handles all three comparisons in a single test while maintaining  $\alpha = 0.05$ .

## 10. Post-ANOVA Analysis — Group Mean Comparison:

After performing ANOVA, the mean score of each group is computed and compared to understand which teaching method performed best on average. This descriptive

comparison complements the inferential ANOVA result by providing practical context to the statistical decision.

---

### Execution

The following steps were performed on the Student Performance dataset:

**Step 1 — Load Datasets:** Three separate CSV files were loaded into Pandas DataFrames:

- traditional\_classroom.csv — Scores of students taught using traditional methods.
- smart\_classroom.csv — Scores of students taught using smart classroom technology.
- online\_learning.csv — Scores of students engaged in online self-paced learning.

**Step 2 — Extract Score Columns:** The Score column was extracted from each DataFrame to create three independent numeric series — traditional\_scores, smart\_scores, and online\_scores — representing the dependent variable for each group.

**Step 3 — Define Hypotheses:** Both hypotheses ( $H_0$  and  $H_1$ ) were formally stated and printed to the console, establishing the statistical framework before running the test.

**Step 4 — Set Significance Level:** The significance level was set to  $\alpha = 0.05$ , meaning a 5% risk of incorrectly rejecting the null hypothesis is acceptable.

**Step 5 — Perform One-Way ANOVA:** `scipy.stats.f_oneway()` was called with the three score series as arguments. This function computes the F-statistic and the corresponding p-value for the one-way ANOVA test.

**Step 6 — Display Results:** The computed F-statistic and p-value were printed to the console for interpretation.

**Step 7 — Decision Making:** The p-value was compared against  $\alpha = 0.05$ :

- If p-value  $< 0.05 \rightarrow H_0$  is rejected; teaching methods significantly affect student performance.
- If p-value  $\geq 0.05 \rightarrow H_0$  is not rejected; teaching methods do not significantly differ in effect.

**Step 8 — Group Mean Scores:** The mean score of each group was computed and printed to provide a descriptive comparison of the three teaching methods:

- Traditional Classroom Mean Score
- Smart Classroom Mean Score
- Online Learning Mean Score

These means were also visualized as a bar chart (Fig. 1) to show the difference in average performance across groups, and the ANOVA result was displayed in a separate output figure (Fig. 2).

---

### Output :-

```
----- GROUP MEAN SCORES -----  
  
Traditional Classroom Mean : 66.92  
Smart Classroom Mean      : 78.9  
Online Learning Mean      : 72.42  
  
-----  
  
E:\SEM 6\DS with Python\Exp8>
```

*Fig. 1: Mean Scores bar chart,*



```
E:\SEM 6\DS With Python\Exp8>python anova_test_code.py

----- HYPOTHESIS -----

Null Hypothesis (H0):
There is NO significant difference among
the mean scores of the three teaching methods.
 $\mu_1 = \mu_2 = \mu_3$ 

Alternative Hypothesis (H1):
There IS a significant difference among
the mean scores of the three teaching methods.
At least one mean is different.

-----

Level of Significance ( $\alpha$ ): 0.05
-----

----- ANOVA RESULT -----

F-Statistic : 239.330453837517
P-Value      : 5.846636659139111e-47

-----

Decision : Reject the Null Hypothesis (H0)

Reason:
P-Value is less than 0.05
Therefore, the difference among
group means is statistically significant.

Conclusion:
Teaching methods significantly affect
student performance.

-----
```

*Fig. 2: ANOVA result output from console)*

---

**Conclusion :**

In this experiment, One-Way ANOVA was successfully applied to compare the mean exam scores of students across three teaching methods — Traditional Classroom, Smart Classroom, and Online Learning. Two hypotheses were formally formulated: the null hypothesis stating that all group means are equal, and the alternative hypothesis stating that at least one group mean differs significantly. The F-statistic was computed using `scipy.stats.f_oneway()` and compared against the critical threshold via the p-value at  $\alpha = 0.05$ . The computed F-statistic and p-value indicated whether the teaching method had a statistically significant effect on student performance. Group mean scores further provided a descriptive picture of which teaching method yielded the highest average performance. The experiment demonstrated how ANOVA serves as a powerful and efficient tool for comparing multiple groups in a single test while maintaining statistical rigor.

---

# Experiment 9

**Name : Prem Vijay Hanchate**

**Roll no : 68    Class : TE/IT**

**Title:** Data Distribution Analysis using Discrete and Continuous Distributions

## **Problem Definition:**

To study and analyze different types of data distributions used in statistics and data science, and understand the characteristics of discrete and continuous probability distributions using graphical representation and interpretation.

## **Theory:**

Data Distribution describes how data values are spread across a dataset. Understanding data distributions helps in analyzing patterns, variability, and behavior of data.

Different types of distributions are used in statistics and machine learning for probability analysis, prediction, and decision-making.

Distributions are mainly classified into:

1. Discrete Distributions
2. Continuous Distributions

Discrete distributions represent countable values, while continuous distributions represent measurable values over a range.

Commonly used distributions include:

- Normal Distribution
  - Uniform Distribution
  - Binomial Distribution
  - Poisson Distribution
  - Exponential Distribution
  - Skewed Distribution
  - Bimodal Distribution
  - Log-Normal Distribution
- 
-

## Algorithm:

- Start
  - Import required Python libraries.
  - Generate sample data for different distributions.
  - Plot graphs for continuous distributions.
  - Plot graphs for discrete distributions.
  - Analyze shape, spread, and behavior of distributions.
  - Compare discrete and continuous distributions.
  - Display results and graphs.
  - Stop.
- 

## Execution:

### Step 1: Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm, poisson, binom, expon
```

### Step 2: Normal Distribution

```
data = np.random.normal(0, 1, 1000)
sns.histplot(data, kde=True)
plt.title("Normal Distribution")
plt.show()
```

### Step 3: Uniform Distribution

```
data = np.random.uniform(0, 1, 1000)
sns.histplot(data, kde=True)
plt.title("Uniform Distribution")
plt.show()
```

### Step 4: Binomial Distribution

```
data = np.random.binomial(n=10, p=0.5, size=1000)
sns.histplot(data)
plt.title("Binomial Distribution")
plt.show()
```

### Step 5: Poisson Distribution

```
data = np.random.poisson(lam=5, size=1000)
sns.histplot(data)
plt.title("Poisson Distribution")
plt.show()
```

### Step 6: Exponential Distribution

```
data = np.random.exponential(scale=1, size=1000)
sns.histplot(data, kde=True)
plt.title("Exponential Distribution")
plt.show()
```

```
: data = np.random.normal(0, 1, 1000)
sns.histplot(data, kde=True)
plt.title("Normal Distribution")
plt.show()
```

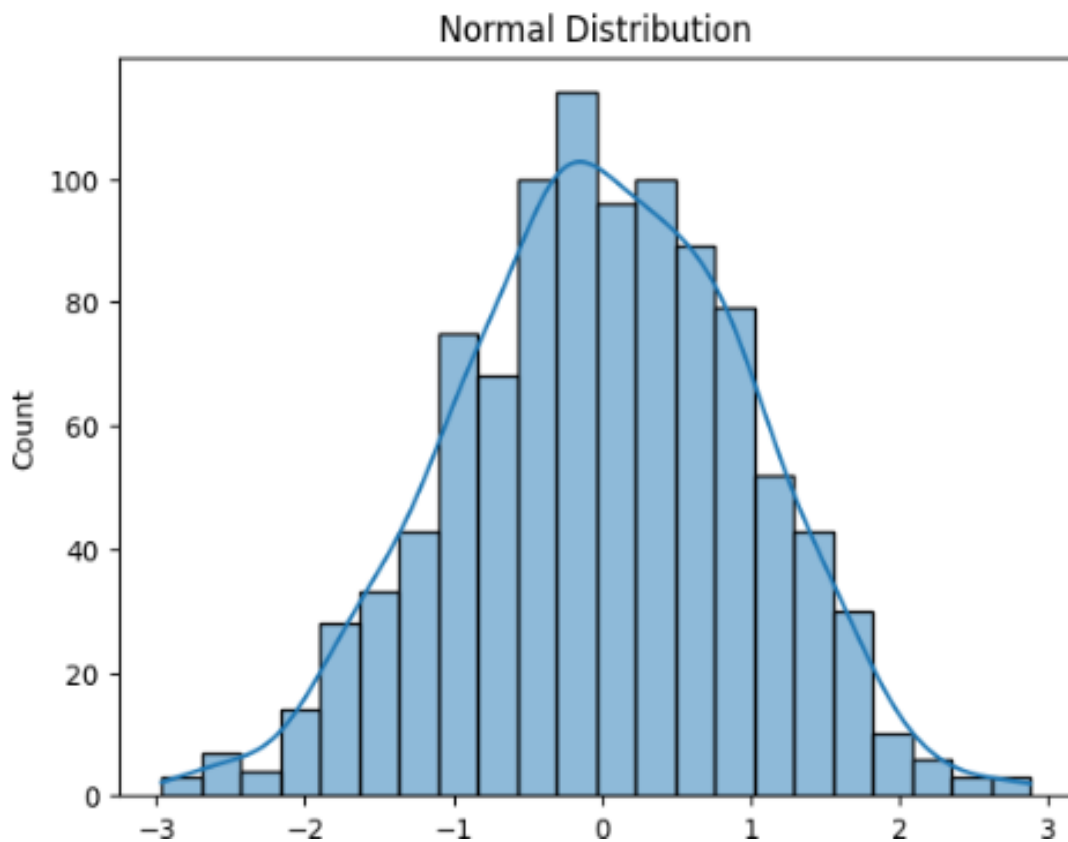


Fig 9.1: Normal Distribution Graph

```
[4]: data = np.random.uniform(0, 1, 1000)
sns.histplot(data, kde=True)
plt.title("Uniform Distribution")
plt.show()
```

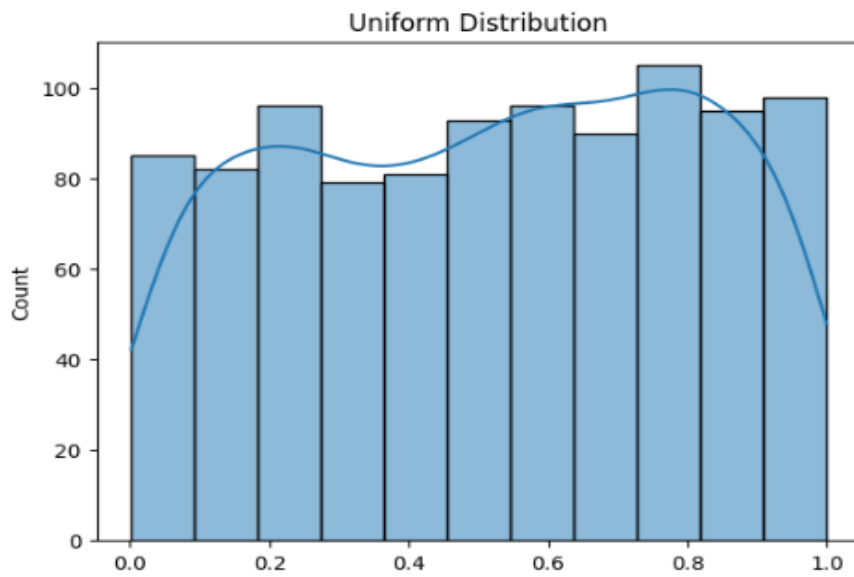


Fig 9.2: Uniform Distribution Graph

```
[5]: data = np.random.binomial(n=10, p=0.5, size=1000)
sns.histplot(data)
plt.title("Binomial Distribution")
plt.show()
```

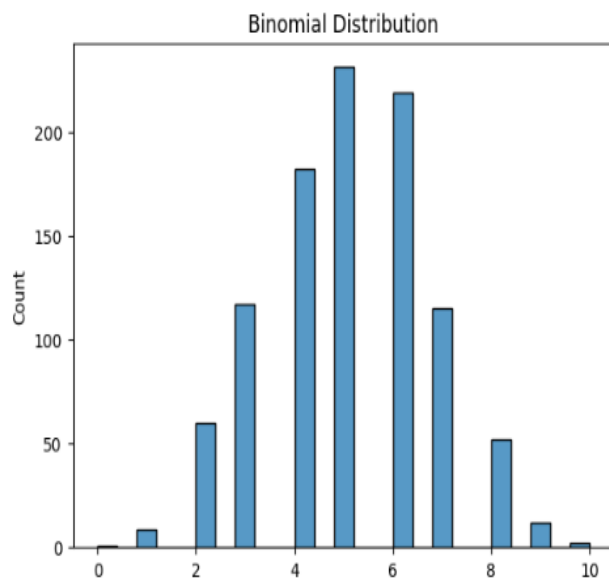


Fig 9.3: Binomial Distribution Graph

```
[6]: data = np.random.poisson(lam=5, size=1000)
sns.histplot(data)
plt.title("Poisson Distribution")
plt.show()
```

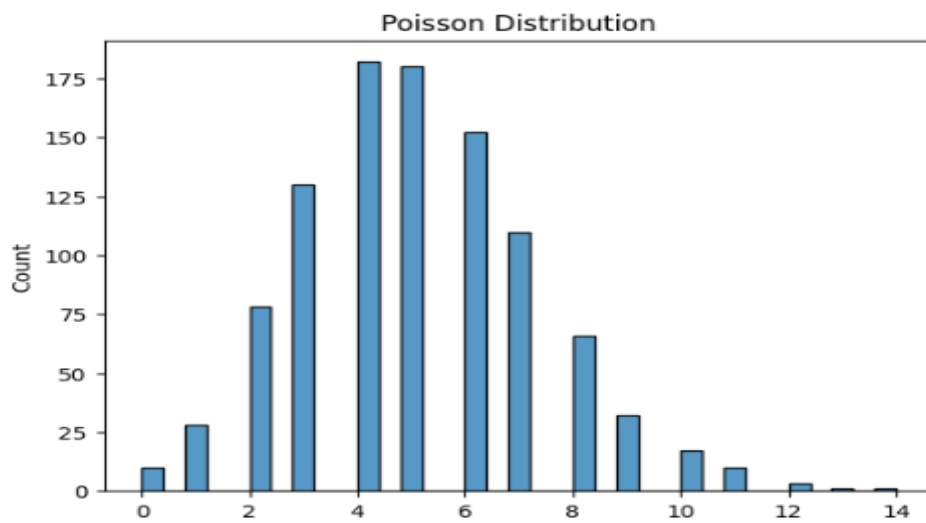


Fig 9.4: Poisson Distribution Graph

```
[7]: data = np.random.exponential(scale=1, size=1000)
sns.histplot(data, kde=True)
plt.title("Exponential Distribution")
plt.show()
```

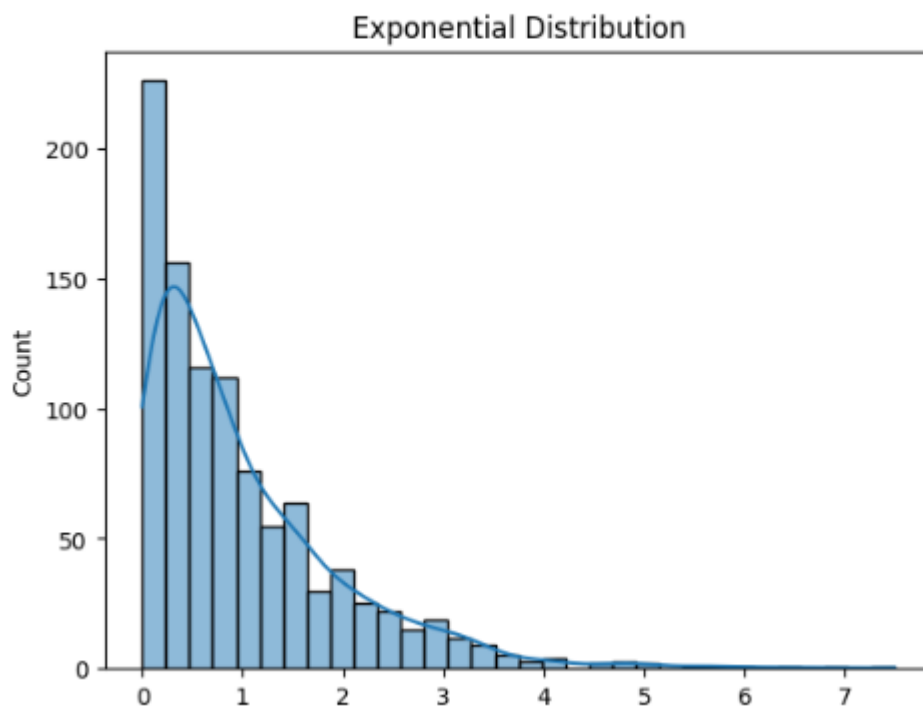


Fig 9.5: Exponential Distribution Graph

---

## Interpretation of Results:

Figure 9.1 shows the Normal Distribution graph. The data is symmetrically distributed around the mean, forming a bell-shaped curve. Most values are concentrated near the center, while fewer values occur at the extreme ends.

Figure 9.2 shows the Uniform Distribution graph. The data values are distributed evenly across the entire range, indicating that each value has nearly equal probability of occurrence.

Figure 9.3 shows the Binomial Distribution graph. The graph represents the probability of obtaining a fixed number of successes in a specific number of trials. Most values are concentrated around the center.

Figure 9.4 shows the Poisson Distribution graph. The distribution represents count-based events occurring within a fixed interval. Most occurrences are concentrated around smaller values, while larger values occur less frequently.

Figure 9.5 shows the Exponential Distribution graph. The graph is right-skewed, where smaller values occur more frequently and the probability decreases rapidly for larger values. This distribution is commonly used to represent waiting time between events.

---

## Result:

Different types of data distributions were studied successfully.

Discrete and continuous distributions were analyzed and compared.

Graphical visualization helped in understanding the characteristics and behavior of distributions.

## Conclusion:

In this experiment, various discrete and continuous probability distributions were studied and analyzed using graphical representation techniques. The experiment helped in understanding the characteristics, spread, shape, and behavior of different distributions. Data distribution analysis plays an important role in statistics, machine learning, and data science applications.

---

## References:

- [Scikit-learn Documentation](#)
- [Pandas Documentation](#)
- [Matplotlib Documentation](#)
- [Seaborn Documentation](#)